

Android FBE EXT4 and Masterkey Recovery Fix

Tool Error reporting and fixing it

Abstract:

Android forensics has been one of the most developing fields in Digital Forensics and Incident Response (DFIR) in recent years where analysis, recovery as well as investigation has always been hard unlike other operating systems. After the introduction of file based encryption in android it became even harder to analyse as well as recover data that is crucial for investigations. For quite some years cold boot attacks have been known to break software based disk encryptions like full disk encryption when an attacker has physical access to a device which is powered on. Raw memory images are easily captured by resetting and rebooting the device with malicious bootloaders. The tools used to break android FDE became ineffective after FBE was made as the default encryption standard for all android devices. To fix this ineffectiveness, a research team under Dr Tobias Gross made a [tool](#) that can successfully recover the master keys in memory of a cold-booted android and was able to decrypt the FBE EXT4 partition and it has been serving as the only open source and effective tool to break FBE till date. But, unfortunately there is a segmentation fault in the extended sleuth kit which makes the whole tool futile. It also had an error in the FBE key finder which we fixed and made the whole tool effective again and tested it on an actual capture of a mobile device.

Introduction:

Android is a mobile operating system built with customization on linux-kernel and other open source softwares. It is mainly designed for smartphones with touch-screen and several other advanced features. It was first created with a goal of operating systems for digital control in 2003. But later google acquired Android Incorporated recognizing its potential in building a versatile stable and efficient operating system for mobile phones. The android 1.0 was released in 2008 with very less security and reliability. But now we are very close to getting Android 15 with so many security and other important features.

Starting with Android 1.0 they include sandboxing applications to restrict interference in the memory space. And in Android 4.0 they improved the application sandboxing, address space layout memory randomization and SELinux enforcing mode for improved security.

From Android 5.0 android secures the disk partition using a new scheme called as the full disk encryption (FDE) which will be enabled by default although it was introduced in Android 3.0. Full Disk Encryption uses a single masterkey to encrypt the entire user data partition and this key is stored in the device's trusted execution environment (TEE) which is protected by the user's general authentication/screen lock. Full Disk Encryption works in a way that the whole data partition remains encrypted until the user unlocks the screen for the first time. Once unlocked the data partition will be decrypted and all files can be accessed. FDE offers strong security for the whole partition but it is useless when the device is running and unlocked. FDE didn't have any granularity.

From Android 7.0 first released in 2016 Android introduced the new security encryption scheme for the user data called File Based Encryption (FBE). It was only made default in Android 10 which was first released in 2019.

File Based Encryption encrypts different files with different keys that can be unlocked independently. FBE divides data into device encrypted and credential encrypted. Devices encrypted are the files that are used by the system and apps that function before the user authenticates. Credential Encrypted are other files which do not fall under the previous domain. FBE has better granularity as it provides better control over access to specific data. Android has been using FBE since its introduction till date.

Methodology for breaking FBE:

FBE was still vulnerable when the attacker had physical access to the device. The cold-boot attack is based on the remanence effect of DRAM which tells us that memory modules have their content preserved for a short time after power blackout. This time of content preservation can be extended by cooling the RAM modules either by cooling sprays or freezing the device. After cooling, two types of cold-boot attack are frequently used. One, by resetting and booting the device with the forensic bootloader to recover encryption keys from RAM, on the other method the power is cut for several seconds and recovery depends on the memory module's temperature.

In DDR3 memory scrambling was introduced which rendered cold-boot attacks difficult because of LFSR based algorithms to encrypt RAM but it was just equal distribution of binary for increased efficiency. But there were also methods introduced to break memory scrambling and memory encryption gradually slows down the system performance by a greater magnitude so it always stayed as a concept and never made it to real world devices that are used by the public. So the tool is built to break android FBE by using raw memory image of a cold-booted android device where master keys can be recovered and the ext4 file system partition encrypted with FBE can be decrypted. This tool can be used until the latest versions of android where KDF has been reconfigured on the hardware making no change in the KDF on any old device update. This tool can be used for most android devices which are still in use.

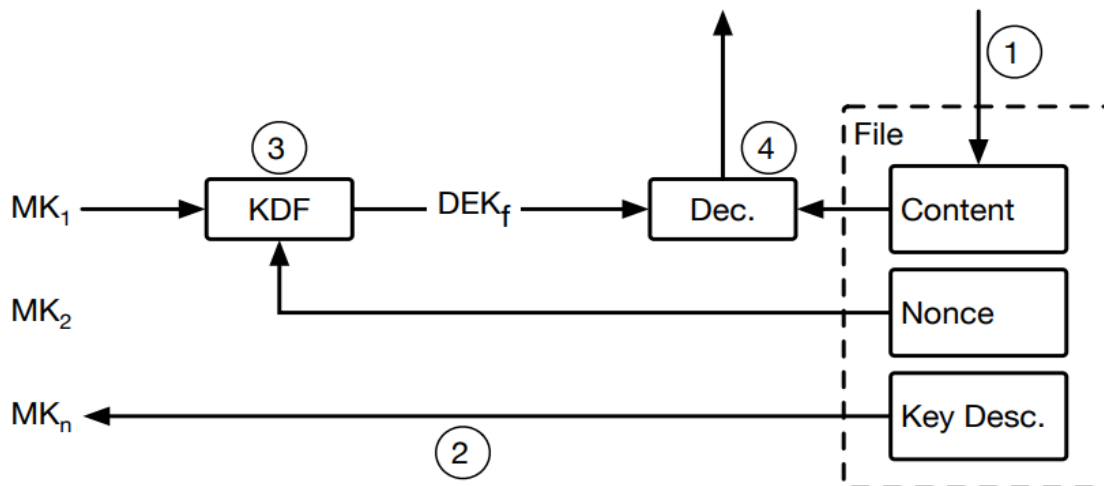
The master keys are recovered from RAM using `aeskeyfind` built by Halderman which is based on algorithm that identifies AES key schedule structure in memory but this tool is not enough to get all the FBE master keys without testing for all decryption and its attributes in the ext4 FBE filesystem. The decryption mechanism was easily possible due to a flaw in the KDF and encryption attributes present in the filesystem as extended attributes without any encoding or encryption. Once we recover the masterkeys, we can choose the correct masterkey for a particular inode with the key descriptor in the extended attributes and the decryption key for the method of encryption given in the extended attributes can be obtained by a simple AES ECB encryption on the masterkey using the file nonce which is also present in the extended attributes. This method is called the key derivation function(KDF). The whole process of content as well as filename decryption is given in a simpler image for better understanding. The decryption key is called as DEK

```

static int derive_key_aes ( u8 deriving_key [FS_AES_128_ECB_KEY_SIZE] ,
const struct fscrypt_key * source_key , u8 derived_raw_key [FS_MAX_KEY_SIZE] )
{
    /* ... */
    struct crypto_skcipher *tfm = crypto_alloc_skcipher( " ecb ( aes ) " , 0 , 0 ) ;
    /* ... */
    res = crypto_skcipher_setkey( tfm , deriving_key , FS_AES_128_ECB_KEY_SIZE ) ;
    /* ... */
    sg_init_one ( &src_sg , source_key->raw , source_key->size ) ;
    sg_init_one ( &dst_sg , derived_raw_key , source_key->size ) ;
    skcipher_request_set_crypt( req , &src_sg , &dst_sg , source_key->size , NULL ) ;
    res = crypto_wait_req ( crypto_skcipher_encrypt(req) , &wait ) ;
    /* ... */
    return res ;
}

```

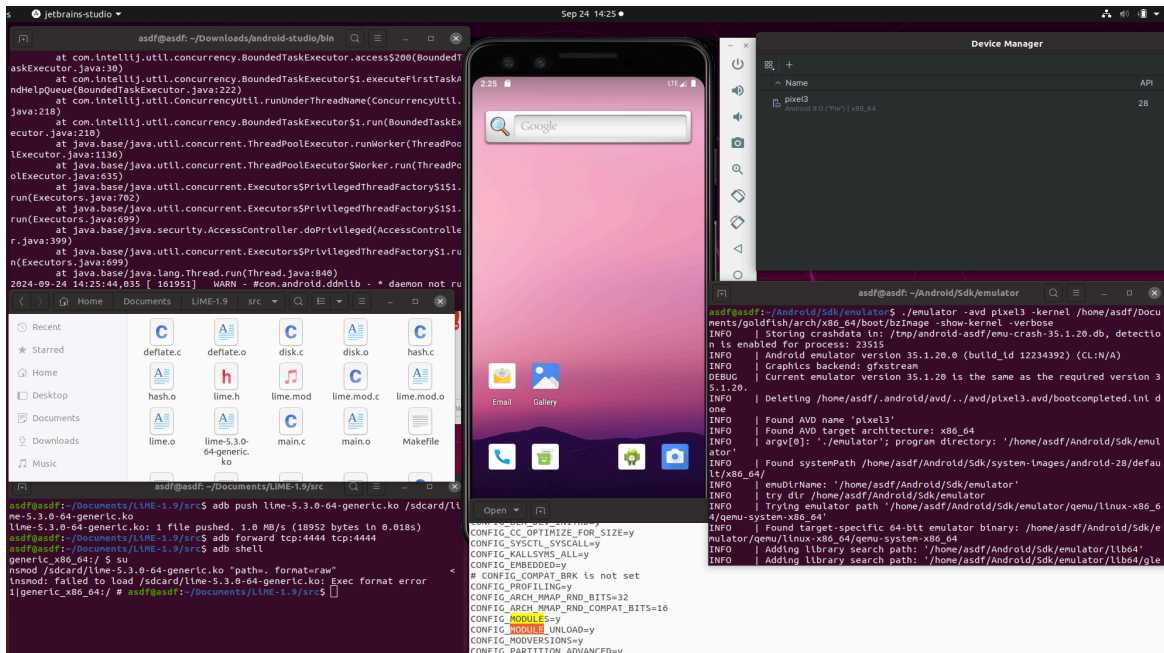
Key Derivation Function(KDF) algorithm



An whole android FBE decryption process

Collection and setup of test data and test environment:

Set up an Ubuntu 19.10 version distro after trying out other distros for a dependable glibc version. Downloaded Android Studio for linux and installed sdk tools and set up an pixel3 AVD image. Also installed android-goldfish-3.10-dev and changed module load parameters in the .config file and custom cross-compiled android kernels. After running the emulator tried to dump memory using LiME(Linux Memory Extractor) but as LiME was deprecated and could not get a raw image file. Instead we got two to three test images from elsewhere which was done with the same process. We were also able to get an actual mobile device test data which is realme3 which is cold-booted and memory is dumped and also we got its encrypted ext4 partition, we used mtkclient utility from bkerler.



Failure of Tool:

First we'll use the original sleuth kit without extended support for decrypting FBE, Using the basic command `fls` to list all the files in the file system

```

joe@rogstrix17: /mnt/l/tech/actual_mobile$ fls -r userdata.img
d/d 11: lost+found
d/d 12: misc
+ d/d 13:
+ r/r 14:
+ d/d 15:
+ r/r 16:
+ d/d 17:
+ d/d 18:
+ r/r 19:
+ d/d 8213:
+ d/d 8214:
+ r/r 21:
+ r/r 22:
+ r/r * 23:
+ d/d 8193:
+ r/r 8194:
+ d/d 8195:
+ r/r 8196:
+ r/r 8670:
+ r/r 8668:
+ d/d 8197:
+ d/d 8198:
+ d/d 8199:
+ d/d 8200:
+ d/d 8261:
+ r/r 8650:
+ r/r 8651:
+ r/r * 8667(realloc):
+ r/r * 8668(realloc):
+ r/r 8238:
+ r/r 8239:
+ r/r 8669:
+ r/r * 8262(realloc):
+ r/r 8262:
+ d/d 8201:
+ r/r 8665:
+ r/r 8237:
+ r/r 8675:
+ d/d 8202:
+ d/d 8203:
+ r/r 8263:
+ r/r 8264:
+ d/d 8205:
+ d/d 8206:
+ d/d 8207:
+ d/d 8208:
+ d/d 8209:
+ d/d 8210:
+ d/d 8211:
+ r/r 8649:
+ r/r 8648:
+ r/r * 8648(realloc):
+ d/d 8212:
+ d/d 8215:
+ d/d 8216:
+ d/d 8217:
+ d/d 8260:
+ d/d 8218:

```

```
joe@rogstrix17:/mnt/l/tech/actual_mobile$ fls -r test_userdata.img
d/d 11: lost+found
r/r 12: local.prop
d/d 13: misc
+ d/d 14:      adb
++ r/r 15:      adb_keys
++ r/r 29:      adb_temp_keys.xml
++ r/r * 30:     adb_temp_keys.xml.bak
+ d/d 16:      apns
++ r/r 17:      apns-conf.xml
+ d/d 18:      pstore
+ d/d 19:      wifi
++ r/r 20:      WifiConfigStore.xml
++ d/d 8213:     sockets
++ d/d 8214:     wpa_supplicant
++ r/r 27:      softap.conf
++ r/r 28:      WifiConfigStore.xml.encrypted-checksum
++ r/r * 30:     WifiConfigStore.xml.bak
+ d/d 8193:     recovery
++ r/r 8194:     ro.build.fingerprint
++ d/d 8195:     proc
+++ r/r 8196:    version
+++ r/r 8773:    version.1
++ r/r 8772:     ro.build.fingerprint.1
+ d/d 8197:     bluedroid
+ d/d 8198:     bluetooth
++ d/d 8199:     logs
+ d/d 8200:     keystore
++ d/d 8273:     user_0
+++ r/r 8734:    1000_USRPKEY_WifiConfigStore+^xml+^encrypted+checksum+^data+integrity+checker+key
+++ r/r 8735:    .1000_chr_USRPKEY_WifiConfigStore+^xml+^encrypted+checksum+^data+integrity+checker+key
+++ r/r 8749:    10098_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+9125194382411554698
+++ r/r 8750:    .10098_chr_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+9125194382411554698
+++ r/r 8752:    10098_USRCERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+9125194382411554698
+++ r/r 8753:    10098_CACERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+9125194382411554698
+++ r/r 8754:    10098_USRPKEY_unstable+^825349ac930c2370b39f30e7d789963b+^2
+++ r/r 8755:    .10098_chr_USRPKEY_unstable+^825349ac930c2370b39f30e7d789963b+^2
+++ r/r 8756:    10098_USRCERT_unstable+^825349ac930c2370b39f30e7d789963b+^2
+++ r/r 8759:    10098_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+6327927474611787316
+++ r/r 8760:    .10098_chr_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+6327927474611787316
+++ r/r 8761:    10098_USRCERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+6327927474611787316
+++ r/r 8762:    10098_CACERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+6327927474611787316
+++ r/r 8763:    10098_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+459843313396393351
+++ r/r 8764:    .10098_chr_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+459843313396393351
+++ r/r 8765:    10098_USRCERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+459843313396393351
+++ r/r 8766:    10098_CACERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+459843313396393351
+++ r/r 8767:    10098_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7178485684714332236
+++ r/r 8768:    .10098_chr_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7178485684714332236
+++ r/r 8769:    10098_USRCERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7178485684714332236
+++ r/r 8770:    10098_CACERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7178485684714332236
+++ r/r 8778:    10098_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7736739232466186306
+++ r/r 8779:    .10098_chr_USRPKEY_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7736739232466186306
+++ r/r 8780:    10098_USRCERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7736739232466186306
+++ r/r 8781:    10098_CACERT_unstable+^c7202ece89390c490b1b94d5b71225e1+^+7736739232466186306
++ r/r 8751:     timestamp
++ r/r 8274:     .metadata
+ d/d 8201:     gatekeeper
++ r/r 8733:     .coldboot
+ d/d 8202:     keychain
++ r/r * 8758(realloc): journal5134528192337530900
```

```

joe@rogstrix17:/mnt/l/tech/actual_mobile$ fls -r userdata.realm.dsk
d/d 11: lost+found
d/d 261633: reserve
+ r/r 261634: ArabicPack.apk
+ r/r 261635: com.realmecomm.app.apk
+ r/r 261636: BengaliPack.apk
+ r/r 261637: EnglishgbPack.apk
+ r/r 261638: Facebook.apk
+ r/r 261639: FrenchPack.apk
+ r/r 261640: KazakhPack.apk
+ r/r 261641: KhmerPack.apk
+ r/r 261642: LaoPack.apk
+ r/r 261643: NepaliPack.apk
+ r/r 261644: PinyinPack.apk
+ r/r 261645: RussianPack.apk
+ r/r 261646: SinhalaPack.apk
+ r/r 261647: SkinPack0DefaultWhite#20180222131233.apk
+ r/r 261648: SwahiliPack.apk
+ r/r 261649: TagalogPack.apk
+ r/r 261650: TamilPack.apk
+ r/r 261651: TouchPal#6.2.6.7.20190113165305_Global_OEM.aligned.apk
+ r/r 261652: UrduPack.apk
+ r/r 261653: com.opera.browser.apk
+ r/r 261654: com.opera.preinstall.apk
+ r/r 261655: com.yandex.browser.apk
+ r/r 261656: ru.yandex.searchplugin.apk
d/d 130817: gr
d/d 915713: etc
+ d/d 915757: recovery
++ r/r 915758: whitelist_recovery_name
++ r/r 915759: whitelist_recovery_path
+ d/d 915715: appchannel
++ r/r 915716: pre_install.appsflyer
++ r/r 915717: OPPO_36577_browserconfiginfo
++ r/r 915718: preinstall-oppo.txt
++ r/r 915719: caih_channel.txt
++ r/r 915720: kwai_vn.appsflyer
++ r/r 915721: dailyhunt_preburn_oppo.txt
++ r/r 915722: zalo_appchannel.in
++ r/r 915723: vova_appchannel.txt
++ r/r 915724: kwai_lite_vn.appsflyer
++ r/r 915725: paytm_preburn_oppo.txt
++ r/r 915726: webnovel_appchannel.txt
++ r/r 915727: qiyioemkey.ini
++ r/r 915728: Newspoint_pre_install.appsflyer
++ r/r 915729: jtexpress_oppo_preburn.ini
++ r/r 915730: wps.txt
++ r/r 915731: shopChannel.txt
++ r/r 915732: lazadaChannel.txt
++ r/r 915733: qdtraditional_preinstall.txt
++ r/r 915734: darazChannel.txt
++ r/r 915735: pre_install.topnews
d/d 392449: app
+ d/d 392450: OneKeyLockScreen
++ r/r 392451: OneKeyLockScreen.apk
++ d/d 392565: lib
++ d/d 392930: oat
+++ d/d 392931: arm64
++++ r/r 392932: OneKeyLockScreen.odex
++++ r/r 393048: OneKeyLockScreen.vdex

```


But when we tested on the actual realme3's disk userdata.realme.dsk we were able to find few files with the inode 0.

```
+ r/r 785036: persistpath_common_JP_ATT.txt
+ r/r 785040: persistname_19662_IN_common.txt
+ r/r 785079: persistname_18325_JP_TELCEL.txt
+ r/r 785086: persistpath_18541_JP_TELCEL.txt
+ r/r 785100: persistpath_18543_JP_OPTUS_PREPAID.txt
+ r/r 785105: persistname_18545_JP_OPTUS.txt
+ r/r 785116: persistpath_18322_JP_TELCEL.txt
+ r/r 785120: persistname_18540_JP_CPW.txt
+ r/r 785122: persistname_18546_JP_ATT.txt
+ r/r 785125: persistpath_18545_JP_VODAFONE_NONEEA.txt
+ r/r 785129: persistname_18321_JP_OPTUS_POSTPAID.txt
+ r/r 785141: persistname_18540_JP_OPTUS_POSTPAID.txt
+ r/r 785160: persistname_18542_JP_ATT.txt
+ r/r 785162: persistname_18326_JP_VODAFONE_NONEEA.txt
+ r/r 785166: persistname_18546_JP_VODAFONE_EEA.txt
+ r/r 785173: persistname_18325_JP_ATT.txt
+ r/r 785176: persistname_18325_JP_CPW.txt
+ r/r 785180: persistname_18547_JP_ATT.txt
+ r/r 785181: persistname_18545_JP_VODAFONE_EEA.txt
+ r/r 785185: persistpath_18546_JP_OPTUS_POSTPAID.txt
+ r/r 785188: persistname_19621_JP_VODAFONE_NONEEA.txt
+ r/r 785189: persistpath_18322_JP_ATT.txt
+ r/r 785194: persistname_18547_JP_OPTUS_PREPAID.txt
+ r/r 785201: persistpath_18543_JP_CPW.txt
+ r/r 785210: persistname_common_EUEX_OPTUS_PREPAID.txt
+ r/r 785220: persistname_18542_JP_OPTUS.txt
+ r/r 785221: persistpath_18321_JP_OPTUS_PREPAID.txt
+ r/r 785242: persistname_18326_JP_VODAFONE_EEA.txt
+ r/r 785253: persistpath_18546_JP_CPW.txt
+ r/- * 0: persistpath_18543_JP_TELCEL.txt
+ r/- * 0: persistpath_18546_JP_VODAFONE_NONEEA.txt
+ r/- * 0: persistpath_18326_JP_OPTUS_PREPAID.txt
+ r/- * 0: persistname_18540_JP_TELCEL.txt
+ r/- * 0: persistname_18546_JP_VODAFONE_NONEEA.txt
+ r/- * 0: persistpath_18540_JP_ATT.txt
+ r/- * 0: persistpath_18321_JP_OPTUS_POSTPAID.txt
+ r/- * 0: persistpath_18542_JP_TELCEL.txt
+ r/- * 0: persistpath_18541_JP_VODAFONE_NONEEA.txt
+ r/- * 0: persistpath_18326_JP_VODAFONE_NONEEA.txt
+ r/- * 0: persistpath_18325_JP_OPTUS_POSTPAID.txt
+ r/- * 0: persistpath_19621_JP_OPTUS_POSTPAID.txt
+ r/- * 0: persistname_18321_JP_VODAFONE_NONEEA.txt
+ r/- * 0: persistpath_18321_JP_VODAFONE_EEA.txt
+ r/- * 0: persistpath_18321_JP_CPW.txt
+ r/- * 0: persistname_18326_JP_OPTUS_PREPAID.txt
+ r/- * 0: persistpath_18542_JP_CPW.txt
+ r/- * 0: persistpath_18542_JP_VODAFONE_EEA.txt
+ r/- * 0: persistpath_18545_JP_CPW.txt
+ r/- * 0: persistname_18541_JP_TELCEL.txt
+ r/r 784898: sd.txt
+ r/r 784902: appchannel_common_BD_common.txt
+ r/r 785001: persistpath_common_MM_common.txt
+ r/r 785006: persistpath_common_PK_common.txt
+ r/r 785007: persistpath_common_RU_common.txt
+ r/r 785008: persistpath_common_SG_TELSTRA.txt
+ r/r 785009: persistpath_common_SG_common.txt
+ r/r 785012: persistpath_common_TW_TELSTRA.txt
+ r/r 785013: persistpath_common_TW_common.txt
+ r/r 785017: persistpathext_common_ID_common.txt
+ r/r 785018: persistpathext_common_IN_common.txt
```

Going forward with compiling the sleuth kit with extended ext4 now let's look into the results of file fls in each disk.

```
azr431knight@Azr431knight:/mnt/e$ fls -r userdata.img | head -5 && echo "-----skip-----" && fls -r userdata.img | tail -5
d/d 11: lost+found
d/d 12: misc
+ d/d 13: adb
++ r/r 14: adb_keys
+ d/d 15: apns
-----skip-----
d/d $ 286721: server_configurable_flags
d/d 294913: rollback-observer
r/r 20: .layout_version
d/d 385025: rollback
V/V 393217: $OrphanFiles
azr431knight@Azr431knight:/mnt/e$ fls -r test_userdata.img | head -5 && echo "-----skip-----" && fls -r test_userdata.img | tail -5
d/d 11: lost+found
r/r 12: local.prop
d/d 13: misc
+ d/d 14: adb
++ r/r 15: adb_keys
-----skip-----
d/d $ 204801: server_configurable_flags
d/d 286721: rollback-observer
r/r 26: .layout_version
d/d 385025: rollback
V/V 393217: $OrphanFiles
```

It works perfectly on the test disks but not on the actual mobile image, as we can see it gives a segmentation fault. But we were able to get the masterkeys for the test ext4 images from its memory dump. So we will also try for this.

```

+ r/r 785105: persistname_-18545_-JP_-OPTUS.txt
+ r/r 785116: persistpath_-18322_-JP_-TELCEL.txt
+ r/r 785120: persistname_-18540_-JP_-CPW.txt
+ r/r 785122: persistname_-18546_-JP_-ATT.txt
+ r/r 785125: persistpath_-18545_-JP_-VODAFONE_NONEEA.txt
+ r/r 785129: persistname_-18321_-JP_-OPTUS_POSTPAID.txt
+ r/r 785141: persistname_-18540_-JP_-OPTUS_POSTPAID.txt
+ r/r 785160: persistname_-18542_-JP_-ATT.txt
+ r/r 785162: persistname_-18326_-JP_-VODAFONE_NONEEA.txt
+ r/r 785166: persistname_-18546_-JP_-VODAFONE_EEA.txt
+ r/r 785173: persistname_-18325_-JP_-ATT.txt
+ r/r 785176: persistname_-18325_-JP_-CPW.txt
+ r/r 785180: persistname_-18547_-JP_-ATT.txt
+ r/r 785181: persistname_-18545_-JP_-VODAFONE_EEA.txt
+ r/r 785185: persistpath_-18546_-JP_-OPTUS_POSTPAID.txt
+ r/r 785188: persistname_-19621_-JP_-VODAFONE_EEA.txt
+ r/r 785189: persistpath_-18322_-JP_-ATT.txt
+ r/r 785194: persistname_-18547_-JP_-OPTUS_PREPAID.txt
+ r/r 785201: persistpath_-18543_-JP_-CPW.txt
+ r/r 785210: persistname_common_-EUEX_-OPTUS_PREPAID.txt
+ r/r 785220: persistname_-18542_-JP_-OPTUS.txt
+ r/r 785221: persistpath_-18321_-JP_-OPTUS_PREPAID.txt
+ r/r 785242: persistname_-18326_-JP_-VODAFONE_EEA.txt
+ r/r 785253: persistpath_-18546_-JP_-CPW.txt
Segmentation fault (core dumped)
azr43lkn1ght@Azr43lKn1ght:/mnt/e$ history | tail -1
1228 history | tail -1
azr43lkn1ght@Azr43lKn1ght:/mnt/e$ history | tail -3
1227 fls -r userdatarealm.dsk
1228 history | tail -1
1229 history | tail -3

```

Using fbekeyfind in this fails as

```

azr43lkn1ght@Azr43lKn1ght:/mnt/d/Android-FBE/fbekeyfind$ python3.8 findMasterKeys.py --part /mnt/e/userdatarealm.dsk /mnt/e/dump_dram_0x40000000.bin
Traceback (most recent call last):
  File "findMasterKeys.py", line 390, in <module>
    g_attributes, g_inodes = g_flfs.get_all_encryption_attributes_from_fs()
  File "findMasterKeys.py", line 48, in get_all_encryption_attributes_from_fs
    inodes = self.all_allocated_inodes()
  File "findMasterKeys.py", line 99, in all_allocated_inodes
    fls_output = subprocess.check_output([self.flfs_wrapper, self.target_image], stderr=subprocess.STDOUT)
  File "/usr/local/lib/python3.8/subprocess.py", line 415, in check_output
    return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,
  File "/usr/local/lib/python3.8/subprocess.py", line 516, in run
    raise CalledProcessError(retcode, process.args,
subprocess.CalledProcessError: Command '['/mnt/d/Android-FBE/fbekeyfind/fls_wrapper.sh', '/mnt/e/userdatarealm.dsk']' returned non-zero exit status 139.

```

And this is also due to the fls as fls_wrapper.sh depends on fls

```

#!/usr/bin/env bash
set -euf -o pipefail

if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <path to image>" >&2
    exit 1
fi

image_path=$1

fls -r $1 | grep -o '^\\+*\\s*[rd]/[rd]\\s*\\*\\$*\\s\\w*.'

```


Now we can go ahead and try fixing it as this error makes the whole tool useless and Android FBE masterkey recovery impossible.

Tool Fix:

First we planned to look into strace and ltrace to trace system calls and library calls to interpret where exactly the segmentation fault occurs

```
write(1, "+ r/r 785162:\tpersistname_-18326"... , 58+ r/r 785162:    persistname_-18326_-JP_-VODAFONE_NONEEA.txt
) = 58
write(1, "+ r/r 785166:\tpersistname_-18546"... , 55+ r/r 785166:    persistname_-18546_-JP_-VODAFONE_EEA.txt
) = 55
write(1, "+ r/r 785173:\tpersistname_-18325"... , 46+ r/r 785173:    persistname_-18325_-JP_-ATT.txt
) = 46
write(1, "+ r/r 785176:\tpersistname_-18325"... , 46+ r/r 785176:    persistname_-18325_-JP_-CPW.txt
) = 46
write(1, "+ r/r 785180:\tpersistname_-18547"... , 46+ r/r 785180:    persistname_-18547_-JP_-ATT.txt
) = 46
write(1, "+ r/r 785181:\tpersistname_-18545"... , 55+ r/r 785181:    persistname_-18545_-JP_-VODAFONE_EEA.txt
) = 55
write(1, "+ r/r 785185:\tpersistpath_-18546"... , 57+ r/r 785185:    persistpath_-18546_-JP_-OPTUS_POSTPAID.txt
) = 57
write(1, "+ r/r 785188:\tpersistname_-19621"... , 55+ r/r 785188:    persistname_-19621_-JP_-VODAFONE_EEA.txt
) = 55
write(1, "+ r/r 785189:\tpersistpath_-18322"... , 46+ r/r 785189:    persistpath_-18322_-JP_-ATT.txt
) = 46
write(1, "+ r/r 785194:\tpersistname_-18547"... , 56+ r/r 785194:    persistname_-18547_-JP_-OPTUS_PREPAID.txt
) = 56
write(1, "+ r/r 785201:\tpersistpath_-18543"... , 46+ r/r 785201:    persistpath_-18543_-JP_-CPW.txt
) = 46
write(1, "+ r/r 785210:\tpersistname_common"... , 58+ r/r 785210:    persistname_common_-EUEX_-OPTUS_PREPAID.txt
) = 58
write(1, "+ r/r 785220:\tpersistname_-18542"... , 48+ r/r 785220:    persistname_-18542_-JP_-OPTUS.txt
) = 48
write(1, "+ r/r 785221:\tpersistpath_-18321"... , 56+ r/r 785221:    persistpath_-18321_-JP_-OPTUS_PREPAID.txt
) = 56
write(1, "+ r/r 785242:\tpersistname_-18326"... , 55+ r/r 785242:    persistname_-18326_-JP_-VODAFONE_EEA.txt
) = 55
write(1, "+ r/r 785253:\tpersistpath_-18546"... , 46+ r/r 785253:    persistpath_-18546_-JP_-CPW.txt
) = 46
--- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_MAPERR, si_addr=0x100} ---
+++ killed by SIGSEGV (core dumped) +++
Segmentation fault (core dumped)
```

```
strlen("persistpath_-18546_-JP_-CPW.txt")    = 31
strlen("persistpath_-18546_-JP_-CPW.txt")    = 31
strlen("persistpath_-18546_-JP_-CPW.txt")    = 31
strlen("persistpath_-18546_-JP_-CPW.txt")    = 31
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb90787, 0x7ffe27f94b30) = 31
free(0x56386c4dce20)                          = <void>
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb9008c, 0x7ffe27f94c10+ r/r 785253:    persistpath_-18546_-JP_-CPW.txt
)                                              = 1
free(0x56386c4d6150)                          = <void>
free(0)                                       = <void>
free(0x56386c4d5f80)                          = <void>
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb9144f, 0x7ffe27f94b80) = 1
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb8ff26, 0x7ffe27f94b80) = 1
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb9115e, 0x7ffe27f94b50) = 2
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb90c3e, 0x7ffe27f94b50) = 2
__vfprintf_chk(0x7f2bfeb75c0, 2, 0x56386bb9116c, 0x7ffe27f94b50) = 2
--- SIGSEGV (Segmentation fault) ---
+++ killed by SIGSEGV +++
```

then we planned to debug it using gdb and figure out exactly where the segmentation fault.

```
azr43lknlight@Azr43lKnlight:/mnt/e$ gdb --args fls -r userdatarealme.dsk
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 5 functions added for GDB 15.0.50.20240403-git in 0.00ms using Python engine 3.12
GEF for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 5 functions added for GDB 15.0.50.20240403-git in 0.00ms using Python engine 3.12
Reading symbols from fls...
```

After debugging instruction by instruction and getting to fs_name.c which prints the filenames in fls, we set a breakpoint in tsk_fs_name_print

```
gef> info b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x000055555555e880 in main(int, char**) at fls.cpp:100
          breakpoint already hit 1 time
2        breakpoint keep y  0x00005555555581220 in tsk_fs_name_print at fs_name.c:402
          breakpoint already hit 1 time
```

And until the next iteration, going instruction by instruction again, we are able to figure out what happens as well as the call stack.

```
threads -----
[#0] Id 1, Name: "fls", stopped 0x555555581220 in tsk_fs_name_print (), reason: BREAKPOINT
trace -----
[#0] 0x555555581220 → tsk_fs_name_print(hFile=0x7ffff79f65c0 <_IO_2_1_stdout_>, fs_file=0x5555555615ba0, a_path=0x7fffffc638 "", fs_attr=0x0, print_path=0x0)
[#1] 0x55555557b95f → printit(fs_file=0x5555555615ba0, a_path=0x7fffffc638 "", fs_attr=0x0, fls_data=0x7fffffd6a0)
[#2] 0x55555557bb88 → print_dent_act(fs_file=0x5555555615ba0, a_path=0x7fffffc638 "", ptr=0x7fffffd6a0)
[#3] 0x55555557e707 → tsk_fs_dir_walk_lcl(a_fs=0x5555555611470, a_dinfo=0x7fffffc230, a_addr=0x2, a_flags=(TSK_FS_DIR_WALK_FLAG_ALL
OC | TSK_FS_DIR_WALK_FLAG_UNALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x55555557bad0 <print_dent_act>, a_ptr=0x7fffffd6a0)
[#4] 0x55555557ef59 → tsk_fs_dir_walk(a_fs=0x5555555611470, a_addr=0x2, a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UN
ALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x55555557bad0 <print_dent_act>, a_ptr=0x7fffffd6a0)
[#5] 0x55555557bd4b → tsk_fs_fls(fs=<optimized out>, lclflags=<optimized out>, inode=<optimized out>, flags=<optimized out>, tpre=<o
ptimized out>, skew=<optimized out>)
[#6] 0x5555555ee29 → main(argc=<optimized out>, argv1=<optimized out>)

gef> info b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x000055555555e880 in main(int, char**) at fls.cpp:100
          breakpoint already hit 1 time
2        breakpoint keep y  0x00005555555581220 in tsk_fs_name_print at fs_name.c:402
          breakpoint already hit 2 times
```

From debugging and disassembling instructions and getting to the code just before the segmentation fault, we reached fs_name.c where the code breaks.

The total lines printed just before segmentation fault is 771, so we will have to skip the breakpoint for the count to get to the actual code breakage.

```
+ r/r 785253: persistpath_-18546_-JP_CPW.txt

Breakpoint 2, tsk_fs_name_print (hFile=0x7ffff79f65c0 <_IO_2_1_stdout_>, fs_file=fs_file@entry=0x55555556164e0,
a_path=a_path@entry=0x7fffffc638 "engineermode/", fs=0x5555555611470, fs_attr=fs_attr@entry=0x0, print_path=0x0)
at fs_name.c:402
402 {
[ Legend: Modified register | Code | Heap | Stack | String ]

registers -----
$rax : 0x00005555555611470 → 0x0000000010101010
$rbx : 0xd
$rcx : 0x00005555555611470 → 0x0000000010101010
$rdx : 0x00007fffffc638 → "engineermode/"
$rsp : 0x00007fffffc048 → 0x000055555557b95f → <printit+00df> mov rax, QWORD PTR [rsp+0x48]
$rbp : 0x00007fffffc638 → "engineermode/"
$rsi : 0x000055555556164e0 → 0x0000000011212212
$rdi : 0x00007ffff79f65c0 → 0x00000000fbad2a84
$rip : 0x0000555555581220 → <tsk_fs_name_print+0000> endbr64
$r8 : 0x0
$r9 : 0x0
$r10 : 0x0000555555561cf80 → 0x0000555500348fec
$r11 : 0xb71f5b3a62f1fff
$r12 : 0x00007fffffd6a0 → 0x0000000000000000
$r13 : 0x00005555555f5a40 → 0x00007ffff79f65c0 → 0x00000000fbad2a84
$r14 : 0x000055555556164e0 → 0x0000000011212212
$r15 : 0x0
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack -----
0x00007fffffc048 +0x0000: 0x000055555557b95f → <printit+00df> mov rax, QWORD PTR [rsp+0x48] ← $rsp
0x00007fffffc050 +0x0008: 0x00005555555611620 → 0x0000000000000000
0x00007fffffc058 +0x0010: 0x0000555555561cf80 → 0x0000555500348fec
0x00007fffffc060 +0x0018: 0x0000000000000060 ("?"?)
0x00007fffffc068 +0x0020: 0x00005555555ad1ad → <tsk_release_lock+000d> test eax, eax
0x00007fffffc070 +0x0028: 0x00005555555611470 → 0x0000000010101010
0x00007fffffc078 +0x0030: 0xffffffffffffff60
0x00007fffffc080 +0x0038: 0x0000000000000002
```

Right now, we are into the last iteration just before the segmentation fault, And we can just debug the function starting from here

```

398 void
399 tsk_fs_name_print(FILE * hFile, const TSK_FS_FILE * fs_file,
400 const char *a_path, TSK_FS_INFO * fs, const TSK_FS_ATTR * fs_attr,
401 uint8_t print_path)
→ 402 {
403     /* type of file - based on dentry type */
404     if (fs_file->name->type < TSK_FS_NAME_TYPE_STR_MAX)
405         tsk_fprintf(hFile, "%s/",
406 tsk_fs_name_type_str[fs_file->name->type]);
407     else

```

threads

[#0] Id 1, Name: "fls", stopped 0x555555581220 in tsk_fs_name_print (), reason: BREAKPOINT

trace

[#0] 0x555555581220 → tsk_fs_name_print(hFile=0x7ffff79f65c0 <_IO_2_1_stdout_>, fs_file=0x5555556164e0, a_path=0x7ffffffffffc638 "engineermode/", fs_attr=0x0, print_path=0x0)

[#1] 0x55555557b95f → printit(fs_file=0x5555556164e0, a_path=0x7ffffffffffc638 "engineermode/", fs_attr=0x0, fls_data=0x7ffffffffffd6a0)

[#2] 0x55555557bb88 → print_dent_act(fs_file=0x5555556164e0, a_path=0x7ffffffffffc638 "engineermode/", ptr=0x7ffffffffffd6a0)

[#3] 0x55555557e707 → tsk_fs_dir_walk_lcl(a_fs=0x555555611470, a_dinfo=0x7ffffffffffc230, a_addr=<optimized out>, a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UNALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x5555557bad0 <print_dent_act>, a_ptr=0x7ffffffffffd6a0)

[#4] 0x55555557e9e8 → tsk_fs_dir_walk_lcl(a_fs=0x555555611470, a_dinfo=0x7ffffffffffc230, a_addr=0x2, a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UNALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x5555557bad0 <print_dent_act>, a_ptr=0x7ffffffffffd6a0)

[#5] 0x55555557ef59 → tsk_fs_dir_walk(a_fs=0x555555611470, a_addr=0x2, a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UNALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x5555557bad0 <print_dent_act>, a_ptr=0x7ffffffffffd6a0)

[#6] 0x55555557b44b → tsk_fs_dir_walk(a_fs=<optimized out>, a_dinfo=<optimized out>, a_addr=<optimized out>, a_flags=<optimized out>, a_ptr=<optimized out>, a_ptr=<optimized out>, skew=<optimized out>, lclflags=<optimized out>, inode=<optimized out>, flags=<optimized out>, tpre=<optimized out>)

[#7] 0x5555555ee29 → main(argc=<optimized out>, argv=<optimized out>)

gef> info b

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x000055555555e880	in main(int, char**) at fls.cpp:100
	breakpoint	already hit	1 time		
2	breakpoint	keep	y	0x00005555555581220	in tsk_fs_name_print at fs_name.c:402
	breakpoint	already hit	772 times		
3	breakpoint	keep	y	0x000055555555812ac	in tsk_fs_name_print at fs_name.c:443
	breakpoint	already hit	771 times		

We will look into the disassembly just before the code break.

```

0x0000555555581297 <+119>: xor     eax, eax
0x0000555555581299 <+121>: call   0x555555560230 <tsk_fprintf>
0x000055555558129e <+126>: mov     rax, QWORD PTR [rbp+0x8]
0x00005555555812a2 <+130>: test    BYTE PTR [rax+0x54], 0x2
0x00005555555812a6 <+134>: jne     0x5555555813f2 <tsk_fs_name_print+466>
=> 0x00005555555812ac <+140>: mov     rax, QWORD PTR [rbp+0x10]
0x00005555555812b0 <+144>: cmp     QWORD PTR [rax+0x100], 0x0
0x00005555555812b8 <+152>: je      0x5555555812cb <tsk_fs_name_print+171>
0x00005555555812ba <+154>: lea     rsi, [rip+0x5ceae] # 0x5555555de16f
0x00005555555812c1 <+161>: mov     rdi, rbx
0x00005555555812c4 <+164>: xor     eax, eax
0x00005555555812c6 <+166>: call   0x555555560230 <tsk_fprintf>
0x00005555555812cb <+171>: mov     rax, QWORD PTR [rbp+0x8]
0x00005555555812cf <+175>: lea     rsi, [rip+0x5e128] # 0x5555555df3fe
0x00005555555812d6 <+182>: mov     rdi, rbx
0x00005555555812d9 <+185>: mov     rdx, QWORD PTR [rax+0x28]

```

And these are the register values right there

```

$rax : 0x0
$rbx : 0x00007ffff79f65c0 → 0x00000000fbad2a84
$rcx : 0x0
$rdx : 0x0
$rsp : 0x00007fffffc020 → 0x000000000000000d ("r"? )
$rbp : 0x00005555556164e0 → 0x0000000011212212
$rsi : 0x202a
$rdi : 0x00007fffffbef40 → 0x00007fffffbef70 → 0x00007fffff202a →
0x0000000000000000
$rip : 0x00005555555812ac → <tsk_fs_name_print+008c> mov rax, QWORD PTR
[rbp+0x10]
$r8 : 0x73 $r9 : 0x0 $r10 : 0xffffffff $r11 : 0x0 $r12 : 0x0 $r13 : 0x0
$r14 : 0x00007fffffc638 → "engineermode/"
$r15 : 0x0 $eflags: [zero carry PARITY adjust sign trap INTERRUPT
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

```

Now let's get to the segmentation fault

```
Program received signal SIGSEGV, Segmentation fault.
0x0000555555812b0 in tsk_fs_name_print (hFile=0x7ffff79f65c0 <_IO_2_1_stdout_>,
fs_file=fs_file@entry=0x5555556164e0,
  a_path=a_path@entry=0x7fffffc638 "engineermode/", fs=<optimized out>,
fs_attr=fs_attr@entry=0x0, print_path=0x0)
  at fs_name.c:443
443      if (fs_file->meta->enc_attr_ptr != NULL)
[ Legend: Modified register | Code | Heap | Stack | String ]
```

——— registers ———

```
$rax : 0x0
$rbx : 0x00007ffff79f65c0 → 0x00000000fbad2a84
$rcx : 0x0
$rdx : 0x0
$rsp : 0x00007fffffc020 → 0x000000000000000d ("r"? )
$rbp : 0x00005555556164e0 → 0x0000000011212212
$rsi : 0x202a
$rdi : 0x00007fffffbe40 → 0x00007fffffbe70 → 0x00007fffff202a →
0x0000000000000000
$rip : 0x0000555555812b0 → <tsk_fs_name_print+0090> cmp QWORD PTR
[rax+0x100], 0x0
$r8 : 0x73
$r9 : 0x0
$r10 : 0xffffffff
$r11 : 0x0
$r12 : 0x0
$r13 : 0x0
$r14 : 0x00007fffffc638 → "engineermode/"
$r15 : 0x0
$eflags: [zero carry PARITY adjust sign trap INTERRUPT direction overflow RESUME
virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
```

——— stack ———

```
0x00007fffffc020 | +0x0000: 0x000000000000000d ("r"? ) ← $rsp
0x00007fffffc028 | +0x0008: 0x00007fffffc638 → "engineermode/"
0x00007fffffc030 | +0x0010: 0x00007fffffd6a0 → 0x0000000000000000
0x00007fffffc038 | +0x0018: 0x0000555555f5a40 → 0x00007ffff79f65c0 →
0x00000000fbad2a84
0x00007fffffc040 | +0x0020: 0x00005555556164e0 → 0x0000000011212212
0x00007fffffc048 | +0x0028: 0x00005555557b95f → <printit+00df> mov rax, QWORD
PTR [rsp+0x48]
0x00007fffffc050 | +0x0030: 0x0000555555611620 → 0x0000000000000000
0x00007fffffc058 | +0x0038: 0x000055555561cf80 → 0x0000555000348fec
```

——— code:x86:64 ———

```
0x555555812a2 <tsk_fs_name_print+0082> test BYTE PTR [rax+0x54], 0x2
0x555555812a6 <tsk_fs_name_print+0086> jne 0x555555813f2
<tsk_fs_name_print+466>
0x555555812ac <tsk_fs_name_print+008c> mov rax, QWORD PTR [rbp+0x10]
→ 0x555555812b0 <tsk_fs_name_print+0090> cmp QWORD PTR [rax+0x100], 0x0
0x555555812b8 <tsk_fs_name_print+0098> je 0x555555812cb
<tsk_fs_name_print+171>
```

```

0x5555555812ba <tsk_fs_name_print+009a> lea    rsi, [rip+0x5ceae]    #
0x55555555de16f
0x5555555812c1 <tsk_fs_name_print+00a1> mov    rdi, rbx
0x5555555812c4 <tsk_fs_name_print+00a4> xor    eax, eax
0x5555555812c6 <tsk_fs_name_print+00a6> call   0x555555560230 <tsk_fprintf>

```

—source:fs_name.c+443 —

```

440     if (fs_file->name->flags & TSK_FS_NAME_FLAG_UNALLOC)
441         tsk_fprintf(hFile, "* ");
442
→ 443     if (fs_file->meta->enc_attr_ptr != NULL)
444         tsk_fprintf(hFile, "$ ");
445
446     tsk_fprintf(hFile, "%" PRIuINUM "", fs_file->name->meta_addr);
447
448     /* print the id and type if we have fs_attr (NTFS) */

```

———— threads ————

[#0] Id 1, Name: "fls", stopped 0x5555555812b0 in tsk_fs_name_print (), reason: SIGSEGV

———— trace ————

```

[#0] 0x5555555812b0 → tsk_fs_name_print(hFile=0x7ffff79f65c0 <_IO_2_1_stdout_>,
fs_file=0x55555556164e0, a_path=0x7fffffc638 "engineermode/", fs=<optimized out>,
fs_attr=0x0, print_path=0x0)
[#1] 0x55555557b95f → printit(fs_file=0x55555556164e0, a_path=0x7fffffc638
"engineermode/", fs_attr=0x0, fls_data=0x7fffffd6a0)
[#2] 0x55555557bb88 → print_dent_act(fs_file=0x55555556164e0, a_path=0x7fffffc638
"engineermode/", ptr=0x7fffffd6a0)
[#3] 0x55555557e707 → tsk_fs_dir_walk_lcl(a_fs=0x5555555611470,
a_dinfo=0x7fffffc230, a_addr=<optimized out>,
a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UNALLOC |
TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x55555557bad0 <print_dent_act>,
a_ptr=0x7fffffd6a0)
[#4] 0x55555557e9e8 → tsk_fs_dir_walk_lcl(a_fs=0x5555555611470,
a_dinfo=0x7fffffc230, a_addr=0x2, a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC |
TSK_FS_DIR_WALK_FLAG_UNALLOC | TSK_FS_DIR_WALK_FLAG_RECURSE),
a_action=0x55555557bad0 <print_dent_act>, a_ptr=0x7fffffd6a0)
[#5] 0x55555557ef59 → tsk_fs_dir_walk(a_fs=0x5555555611470, a_addr=0x2,
a_flags=(TSK_FS_DIR_WALK_FLAG_ALLOC | TSK_FS_DIR_WALK_FLAG_UNALLOC |
TSK_FS_DIR_WALK_FLAG_RECURSE), a_action=0x55555557bad0 <print_dent_act>,
a_ptr=0x7fffffd6a0)
[#6] 0x55555557bd4b → tsk_fs_fls(fs=<optimized out>, lclflags=<optimized out>,
inode=<optimized out>, flags=<optimized out>, tpre=<optimized out>, skew=<optimized
out>)
[#7] 0x55555555ee29 → main(argc=<optimized out>, argv1=<optimized out>)

```

Right after the program end , the Program instruction pointer was in 0x5555555812b0
<tsk_fs_name_print+144>: "H\203\270"

Let's audit the source code until there to figure out what exactly is the reason.
Let's go with how callstack worked for easier code audit flow.

```
tools > fstools > C fls.cpp > main(int, char **)
99  main(int argc, char **argv)
434  {
435
436      if (tsk_fs_fls(fs, (TSK_FS_FLS_FLAG_ENUM) fls_flags, inode,
437                  (TSK_FS_DIR_WALK_FLAG_ENUM) name_flags, macpre, sec_skew)) {
438          tsk_error_print(stderr);
439          fs->close(fs);
440          img->close(img);
441          exit(1);
442      }
443
444      fs->close(fs);
445      img->close(img);
446
447      exit(0);
448  }
449
```

Here the main function calls tsk_fs_fls in fls_lib.c

```
tsk > fs > C fls_lib.c > tsk_fs_fls(TSK_FS_INFO *, TSK_FS_FLS_FLAG_ENUM, TSK_INUM_T, TSK_FS_DIR_WALK_FLAG_ENUM, TSK_TCHAR *, int32_t)
208  uint8_t
209  tsk_fs_fls(TSK_FS_INFO * fs, TSK_FS_FLS_FLAG_ENUM lclflags,
210            TSK_INUM_T inode, TSK_FS_DIR_WALK_FLAG_ENUM flags, TSK_TCHAR * tpre,
211            int32_t skew)
212  {
213      FLS_DATA data;
214
215      data.flags = lclflags;
216      data.sec_skew = skew;
217
218      #ifdef TSK_WIN32
219      {
220          UTF8 *ptr8;
221          UTF16 *ptr16;
222          int retval;
223
224          if ((tpre != NULL) && (TSTRLEN(tpre) > 0)) {
225              size_t clen = TSTRLEN(tpre) * 4;
226              data.macpre = (char *) tsk_malloc(clen);
227              if (data.macpre == NULL) {
228                  return 1;
229              }
230              ptr8 = (UTF8 *) data.macpre;
231              ptr16 = (UTF16 *) tpre;
232
233              retval =
234                  tsk_UTF16toUTF8_lclorder((const UTF16 **) &ptr16, (UTF16 *)
235                  & ptr16[TSTRLEN(tpre) + 1], &ptr8,
236                  (UTF8 *) ((uintptr_t) ptr8 + clen), TSKlenientConversion);
237              if (retval != TSKconversionOK) {
238                  tsk_error_reset();
239                  tsk_error_set_errno(TSK_ERR_FS_UNICODE);
240                  tsk_error_set_errstr
241                      ("Error converting fls mactime pre-text to UTF-8 %d\n",
242                      retval);
243                  return 1;
244              }
245          }
246      }
247      #endif
248
249      return tsk_fs_fls_impl(fs, inode, flags, data, skew);
250  }
```

And we have a if else branch that does not affect much of the code flow. So we go forward with what is after the else block.


```

tsk > fs > C fls_lib.c > tsk_fs_fls(TSK_FS_INFO *, TSK_FS_FLS_FLAG_ENUM, TSK_INUM_T, TSK_FS_DIR_WALK_FLAG_ENUM, TSK_TCHAR *, int32_t)
209 tsk_fs_fls(TSK_FS_INFO * fs, TSK_FS_FLS_FLAG_ENUM lclflags,
245 }
246 else {
247     data.macpre = (char *) tsk_malloc(1);
248     if (data.macpre == NULL) {
249         return 1;
250     }
251     data.macpre[0] = '\0';
252 }
253
254 retval = tsk_fs_dir_walk(fs, inode, flags, print_dent_act, &data);
255
256 free(data.macpre);
257 data.macpre = NULL;
258 return retval;
259 }
260 #else
261 data.macpre = tpre;
262 return tsk_fs_dir_walk(fs, inode, flags, print_dent_act, &data);
263 #endif
264 }
265

```

Here the call for `tsk_fs_dir_walk` happens which is in `fs_dir.c`

```

uint8_t
tsk_fs_dir_walk(TSK_FS_INFO * a_fs, TSK_INUM_T a_addr,
    TSK_FS_DIR_WALK_FLAG_ENUM a_flags, TSK_FS_DIR_WALK_CB a_action,
    void *a_ptr)
{
    DENT_DINFO dinfo;
    TSK_WALK_RET_ENUM retval;

    if ((a_fs == NULL) || (a_fs->tag != TSK_FS_INFO_TAG)) {
        tsk_error_set_errno(TSK_ERR_FS_ARG);
        tsk_error_set_errstr
            ("tsk_fs_dir_walk: called with NULL or unallocated
structures");
        return 1;
    }

    memset(&dinfo, 0, sizeof(DENT_DINFO));
    if ((dinfo.stack_seen = tsk_stack_create()) == NULL)
        return 1;

    /* Sanity check on flags -- make sure at least one ALLOC is set
*/
    if (((a_flags & TSK_FS_DIR_WALK_FLAG_ALLOC) == 0) &&
        ((a_flags & TSK_FS_DIR_WALK_FLAG_UNALLOC) == 0)) {
        a_flags |=
            (TSK_FS_DIR_WALK_FLAG_ALLOC |

```

```

TSK_FS_DIR_WALK_FLAG_UNALLOC);
    }

    /* if the flags are right, we can collect info that may be needed
     * for an orphan walk. If the walk fails or stops, the code that
     * calls the action will clear this stuff.
     */
    tsk_take_lock(&a_fs->list_inum_named_lock);
    if ((a_fs->list_inum_named == NULL) && (a_addr ==
a_fs->root_inum)
        && (a_flags & TSK_FS_DIR_WALK_FLAG_RECURSE)) {
        dinfo.save_inum_named = 1;
    }
    tsk_release_lock(&a_fs->list_inum_named_lock);

    retval = tsk_fs_dir_walk_lcl(a_fs, &dinfo, a_addr, a_flags,
        a_action, a_ptr);

    // if we were saving the list of named files in the temp list,
    // then now save them to FS_INFO
    if (dinfo.save_inum_named == 1) {
        if (retval != TSK_WALK_CONT) {
            /* There was an error and we stopped early, so we should
get
            * rid of the partial list we were making.
            */
            tsk_list_free(dinfo.list_inum_named);
            dinfo.list_inum_named = NULL;
        }
        else {
            save_inum_named(a_fs, &dinfo);
        }
    }

    tsk_stack_free(dinfo.stack_seen);

    if (retval == TSK_WALK_ERROR)
        return 1;
    else
        return 0;
}

```

Here the code flow continues to `tsk_fs_dir_walk_lcl` in the same file with recursive calls and then returns to `print_dent_act` in `fls_lib.c`

```
tsk > fs > C fls_lib.c > print_dent_act(TSK_FS_FILE *, const char *, void *)
114 print_dent_act(TSK_FS_FILE * fs_file, const char *a_path, void *ptr)
159     }
160
161     printit(fs_file, a_path, fs_attr, fls_data);
162 }
163     else if (fs_attr->type == TSK_FS_ATTR_TYPE_NTFS_IDXROOT) {
164         printed = 1;
165
166         /* If it is . or .. only print it if the flags say so,
167          * we continue with other streams though in case the
168          * directory has a data stream
169          */
170         if (!((TSK_FS_ISDOT(fs_file->name->name)) &&
171             ((fls_data->flags & TSK_FS_FLS_DOT) == 0)))
172             printit(fs_file, a_path, fs_attr, fls_data);
173     }
174     /* Print the FILE_NAME times if this is the same attribute
175      * that we collected the times from. */
176     else if ((fs_attr->type == TSK_FS_ATTR_TYPE_NTFS_FNAME) &&
177         (fs_attr->id == fs_file->meta->time2.ntfs.fn_id) &&
178         (fls_data->flags & TSK_FS_FLS_MAC)) {
179         /* If it is . or .. only print it if the flags say so,
180          * we continue with other streams though in case the
181          * directory has a data stream
182          */
183         if (!((TSK_FS_ISDOT(fs_file->name->name)) &&
184             ((fls_data->flags & TSK_FS_FLS_DOT) == 0)))
185             printit(fs_file, a_path, fs_attr, fls_data);
186     }
187 }
188
189 /* A user reported that an allocated file had the standard
190  * attributes, but no $Data. We should print something */
191 if (printed == 0) {
192     printit(fs_file, a_path, NULL, fls_data);
193 }
```

Here it has several `printit` calls which is in the same file and which calls the function `tsk_fs_name_print` in `fs_name.c`

```
95     }
96     else if (fls_data->flags & TSK_FS_FLS_LONG) {
97         tsk_fs_name_print_long(stdout, fs_file, a_path, fs_file->fs_info,
98             fs_attr, TSK_FS_FLS_FULL & fls_data->flags ? 1 : 0,
99             fls_data->sec_skew);
100         tsk_printf("\n");
101     }
102     else {
103         tsk_fs_name_print(stdout, fs_file, a_path, fs_file->fs_info,
104             fs_attr, TSK_FS_FLS_FULL & fls_data->flags ? 1 : 0);
105         tsk_printf("\n");
106     }
107 }
108 }
```

Auditing the code with few print statements

```
tsk > fs > C fs_name.c > tsk_fs_name_print(FILE *, const TSK_FS_FILE *, const char *, TSK_FS_INFO *, const TSK_FS_ATTR *, uint8_t)
399 tsk_fs_name_print(FILE * hFile, const TSK_FS_FILE * fs_file,
437
438
439 /* print a * if it is deleted */
440 if ((fs_file->name->flags & TSK_FS_NAME_FLAG_UNALLOC)
441     tsk_fprintf(hFile, "* ");
442
443     printf("[Azr431Kn1ght %ld]", fs_file->meta);
444
445     if (fs_file->meta->enc_attr_ptr != NULL)
446         tsk_fprintf(hFile, "$ ");
447
448     tsk_fprintf(hFile, "% PRIuINUM ", fs_file->name->meta_addr);
449
```

The program does not reach the next line so we will go a line above to print it.
So after some addition of code in fs_lib.c

```
tsk > fs > C fs_lib.c > tsk_fs_fls(TSK_FS_INFO *, TSK_FS_FLS_FLAG_ENUM, TSK_INUM_T, TSK_FS_DIR_WALK_FLAG_ENUM, TSK_TCHAR *, int32_t)
32 typedef struct {
33
34     /*directory prefix for printing mactime output */
35     char *macpre;
36     int flags;
37 } FLS_DATA;
38
39 #include <execinfo.h> #include errors detected. Please update your includePath. Squiggles are disabled
40 #include <signal.h>
41 void handler(int sig) {
42     void *array[10];
43     size_t size;
44
45     // get void*'s for all entries on the stack
46     size = backtrace(array, 10);
47
48     // print out all the frames to stderr
49     fprintf(stderr, "Error: signal %d:\n", sig);
50     backtrace_symbols_fd(array, size, STDERR_FILENO);
51     exit(1);
52 }
53
```

And we are able to figure out that fs_file->meta in fs_name.c is Null which could cause us the segmentation fault, so it's better to look into the directory and the inodes that are owned to it.

First let's look into the structure of EXT4 which will get us to the inode that causes the segmentation fault.

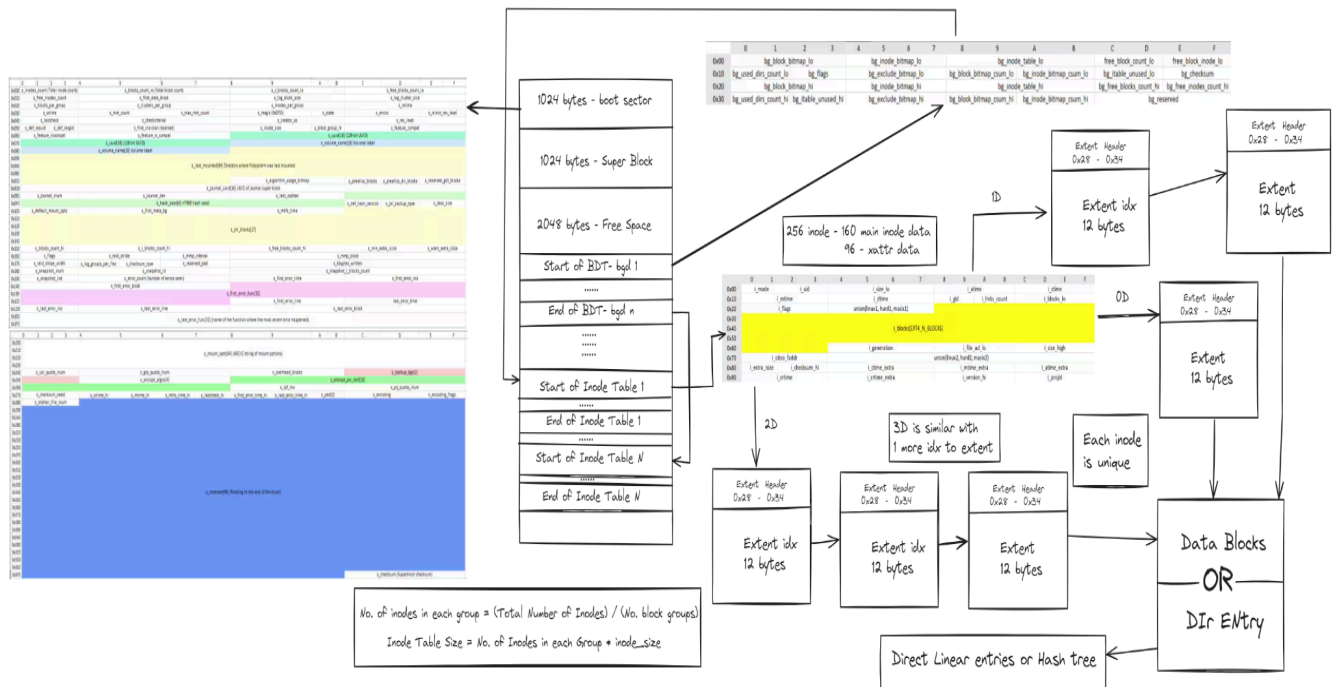
We will use the active@ disk editor for going through the structure easily.

First 1024 bytes is bootsector, next 1024 is superblock which will have all the details of the EXT4 filesystem like size of inode, block group descriptor, inode table, block group descriptor table, block count, etc..

Then we have reserved sectors for the next 2048 bytes and which so far will be a total block to skip. A block in EXT4 is 4096 bytes.

Then we have the block group descriptor table which will be either 32 or 64 bytes depending on the superblock. This will give us the number of inodes used, location of inode table, inode bitmap and block bitmap, etc..

From here we can go to the inode table and we will be able to analyse it but doing it manually for the whole process is hard, so we made an easier illustrative diagram to understand the whole structure and a python parser to make it easier.



And the python parser for the whole EXT4 system is given below. The tool is also uploaded in <https://github.com/Azr43IKn1ght/ext4parser.git> , It is only made dedicated for Tech contest to easily understand EXT4 and fix the tool.

```
import os
import sys
from argparse import ArgumentParser
import math
import time
from pathlib import Path
from datetime import datetime
from rich.console import Console
from rich.progress import Progress, SpinnerColumn, TextColumn
```

```
def banner():
    console.print("""[bold red]\n
```

```

An Ext4 File System Parser
Tool to Parse Linux and Android FileSystems

A Complete Ext4 File System Parser built by Azr43IKn1ght(Nithin Chenthur Prabhu)
https://github.com/Azr43IKn1ght

```

[/bold red]\n""",)

EXT4_BLOCK_SZ = 4096
EXT4_MIN_BLOCK_SIZE = 1024
EXT4_MAX_BLOCK_SIZE = 65536

Ext4 Inode

EXT4_BAD_INO = 1
EXT4_ROOT_INO = 2
EXT4_BOOT_LOADER_INO = 5
EXT4_UNDEL_DIR_INO = 6
EXT4_RESIZE_INO = 7
EXT4_JOURNAL_INO = 8
EXT4_GOOD_OLD_FIRST_INO = 11

Ext4 Super Block

EXT4_SUPER_MAGIC = 0xEF53

EXT4_STATE = {
 'EXT4_VALID_FS' : 0x0001,
 'EXT4_ERROR_FS' : 0x0002,
 'EXT4_ORPHAN_FS' : 0x0004
}

EXT4_ERRORS = {
 'EXT4_ERRORS_CONTINUE' : 1,
 'EXT4_ERRORS_RO' : 2,
 'EXT4_ERRORS_PANIC' : 3
}

EXT4_OS = {
 'EXT4_OS_LINUX' : 0,
 'EXT4_OS_HURD' : 1,
 'EXT4_OS_MASIX' : 2,
 'EXT4_OS_FREEBSD' : 3,
 'EXT4_OS_LITES' : 4
}

EXT4_REV_LEVEL = {
 'EXT4_GOOD_OLD_REV' : 0,
 'EXT4_DYNAMIC_REV' : 1
}

EXT4_DEF_RESERVED_ID = {
 'EXT4_DEF_RESUID' : 0,
 'EXT4_DEF_RESUID' : 0
}

EXT4_INODE_NO = {
 'EXT4_BAD_INO' : 1,
 'EXT4_ROOT_INO' : 2,
 'EXT4_BOOT_LOADER_INO' : 5,
 'EXT4_UNDEL_DIR_INO' : 6,
 'EXT4_RESIZE_INO' : 7,
 'EXT4_JOURNAL_INO' : 8,
 'EXT4_GOOD_OLD_FIRST_INO' : 11
}

EXT4_FEATURE_COMPAT = {
 'EXT4_FEATURE_COMPAT_DIR_PREALLOC' : 0x0001,
 'EXT4_FEATURE_COMPAT_IMAGIC_INODES' : 0x0002,
 'EXT4_FEATURE_COMPAT_HAS_JOURNAL' : 0x0004,
 'EXT4_FEATURE_COMPAT_EXT_ATTR' : 0x0008,
 'EXT4_FEATURE_COMPAT_RESIZE_INODE' : 0x0010,
 'EXT4_FEATURE_COMPAT_DIR_INDEX' : 0x0020
}

EXT4_FEATURE_INCOMPAT = {
 'EXT4_FEATURE_INCOMPAT_COMPRESSION' : 0x0001,
 'EXT4_FEATURE_INCOMPAT_FILETYPE' : 0x0002,
 'EXT4_FEATURE_INCOMPAT_RECOVER' : 0x0004,
 'EXT4_FEATURE_INCOMPAT_JOURNAL_DEV' : 0x0008,
 'EXT4_FEATURE_INCOMPAT_META_BG' : 0x0010,
 'EXT4_FEATURE_INCOMPAT_EXTENTS' : 0x0040,
 'EXT4_FEATURE_INCOMPAT_64BIT' : 0x0080,
 'EXT4_FEATURE_INCOMPAT_MMP' : 0x0100,
}


```

'EXT4_FEATURE_INCOMPAT_FLEX_BG' : 0x0200,
'EXT4_FEATURE_INCOMPAT_EA_INODE' : 0x0400,
'EXT4_FEATURE_INCOMPAT_DIRDATA' : 0x1000
}

EXT4_FEATURE_RO_COMPAT = {
'EXT4_FEATURE_RO_COMPAT_SPARSE_SUPER' : 0x0001,
'EXT4_FEATURE_RO_COMPAT_LARGE_FILE' : 0x0002,
'EXT4_FEATURE_RO_COMPAT_BTREE_DIR' : 0x0004,
'EXT4_FEATURE_RO_COMPAT_HUGE_FILE' : 0x0008,
'EXT4_FEATURE_RO_COMPAT_GDT_CSUM' : 0x0010,
'EXT4_FEATURE_RO_COMPAT_DIR_NLINK' : 0x0020,
'EXT4_FEATURE_RO_COMPAT_EXTRA_ISIZE' : 0x0040
}

EXT4_DEFAULT_MOUNT_OPTS = {
'EXT2_DEFM_DEBUG' : 0x0001,
'EXT2_DEFM_BSDGROUPS' : 0x0002,
'EXT2_DEFM_XATTR_USER' : 0x0004,
'EXT2_DEFM_ACL' : 0x0008,
'EXT2_DEFM_UID16' : 0x0010,
'EXT3_DEFM_JMODE_DATA' : 0x0020,
'EXT3_DEFM_JMODE_ORDERED' : 0x0040,
'EXT3_DEFM_JMODE' : 0x0060,
'EXT3_DEFM_JMODE_WBACK' : 0x0060,
'EXT4_DEFM_NOBARRIER' : 0x0100,
'EXT4_DEFM_BLOCK_VALIDITY' : 0x0200,
'EXT4_DEFM_DISCARD' : 0x0400,
'EXT4_DEFM_NODELALLOC' : 0x0800
}

EXT4_MISC_FLAGS = {
'EXT2_FLAGS_SIGNED_HASH' : 0x0001,
'EXT2_FLAGS_UNSIGNED_HASH' : 0x0002,
'EXT2_FLAGS_TEST_FILESYS' : 0x0004,
'EXT2_FLAGS_IS_SNAPSHOT' : 0x0010,
'EXT2_FLAGS_FIX_SNAPSHOT' : 0x0020,
'EXT2_FLAGS_FIX_EXCLUDE' : 0x0040
}

# Ext4 Block Group Descriptors
EXT4_BG_FLAGS = {
'EXT2_BG_INODE_UNINIT' : 0x0001,
'EXT2_BG_BLOCK_UNINIT' : 0x0002,
'EXT2_BG_INODE_ZEROED' : 0x0004
}

# Ext4 Inode Table
EXT4_INODE_ENTRY_SZ = 128
EXT4_NDIR_BLOCKS = 12
EXT4_IND_BLOCK = EXT4_NDIR_BLOCKS
EXT4_DIND_BLOCK = EXT4_IND_BLOCK + 1
EXT4_TIND_BLOCK = EXT4_DIND_BLOCK + 1
EXT4_N_BLOCKS = EXT4_TIND_BLOCK + 1

EXT4_INODE_MODE = {
'S_IXOTH' : 0x1,
'S_IWOTH' : 0x2,
'S_IROTH' : 0x4,
'S_IXGRP' : 0x8,
'S_IWGRP' : 0x10,
'S_IRGRP' : 0x20,
'S_IXUSR' : 0x40,
'S_IWUSR' : 0x80,
'S_IRUSR' : 0x100,
'S_ISVTX' : 0x200,
'S_ISGID' : 0x400,
'S_ISUID' : 0x800,
# These are mutually-exclusive file types
'S_IFIFO' : 0x1000,
'S_IFCHR' : 0x2000,
'S_IFDIR' : 0x4000,
'S_IFBLK' : 0x6000,
'S_IFREG' : 0x8000,
'S_IFLNK' : 0xA000,

```

```

'S_IFSOCK' : 0xC000
}

EXT4_INODE_FLAGS = {
    'EXT4_SECRM_FL'      : 0x1,
    'EXT4_UNRM_FL'       : 0x2,
    'EXT4_COMPR_FL'       : 0x4,
    'EXT4_SYNC_FL'        : 0x8,
    'EXT4_IMMUTABLE_FL'   : 0x10,
    'EXT4_APPEND_FL'      : 0x20,
    'EXT4_NODUMP_FL'      : 0x40,
    'EXT4_NOATIME_FL'     : 0x80,
    'EXT4_DIRTY_FL'       : 0x100,
    'EXT4_COMPRBLK_FL'    : 0x200,
    'EXT4_NOCOMPR_FL'     : 0x400,
    'EXT4_ECOMPR_FL'      : 0x800,
    'EXT4_INDEX_FL'       : 0x1000,
    'EXT4_IMAGIC_FL'      : 0x2000,
    'EXT4_JOURNAL_DATA_FL' : 0x4000,
    'EXT4_NOTAIL_FL'      : 0x8000,
    'EXT4_DIRSYNC_FL'     : 0x10000,
    'EXT4_TOPDIR_FL'      : 0x20000,
    'EXT4_HUGE_FILE_FL'   : 0x40000,
    'EXT4_EXTENTS_FL'     : 0x80000,
    'EXT4_EA_INODE_FL'    : 0x200000,
    'EXT4_EOFBLOCKS_FL'   : 0x400000,
    'EXT4_RESERVED_FL'   : 0x80000000,
    # Aggregate flags
    'EXT4_FL_USER_VISIBLE' : 0x4BDFFF,
    'EXT4_FL_USER_MODIFIABLE' : 0x4B80FF,
}

# Ext4 Extent Tree
EXT4_EXTENT_TREE_MAGIC = 0xF30A

# Ext4 Directory Entries
EXT4_NAME_LEN = 255
EXT4_HTREE_NAME_LEN = 4
# for hash tree directory resolution
EXT4_FILE_TYPE = {
    'EXT4_FT_UNKNOWN' : 0x0,
    'EXT4_FT_REG_FILE' : 0x1,
    'EXT4_FT_DIR' : 0x2,
    'EXT4_FT_CHRDEV' : 0x3,
    'EXT4_FT_BLKDEV' : 0x4,
    'EXT4_FT_FIFO' : 0x5,
    'EXT4_FT_SOCK' : 0x6,
    'EXT4_FT_SYMLINK' : 0x7
}

# Ext4 Extended Attributes
EXT4_XATTR_MAGIC = 0xEA020000
# Journal will be implemented in next version
# Ext4 Journal, jbd2
EXT4_JNL_BACKUP_BLOCKS = 1
JBD2_MAGIC_NUMBER = 0xC03B3998
# JBD2_CHECKSUM_BYTES = (32 / sizeof(u32))
JBD2_CHECKSUM_BYTES = (32 / 4)

EXT4_JNL_BLOCK_TYPE = {
    'JBD2_DESCRIPTOR_BLOCK' : 1,
    'JBD2_COMMIT_BLOCK' : 2,
    'JBD2_SUPERBLOCK_V1' : 3,
    'JBD2_SUPERBLOCK_V2' : 4,
    'JBD2_REVOKE_BLOCK' : 5
}

EXT4_JNL_FEATURE_COMPAT = {
    'JBD2_FEATURE_COMPAT_CHECKSUM' : 0x00000001
}

EXT4_JNL_FEATURE_INCOMPAT = {
    'JBD2_FEATURE_INCOMPAT_REVOKE' : 0x00000001,
    'JBD2_FEATURE_INCOMPAT_64BIT' : 0x00000002,
    'JBD2_FEATURE_INCOMPAT_ASYNC_COMMIT' : 0x00000004
}

```

```

EXT4_JNL_FLAGS = {
    'JBD2_FLAG_ESCAPE' : 1,
    'JBD2_FLAG_SAME_UUID' : 2,
    'JBD2_FLAG_DELETED' : 4,
    'JBD2_FLAG_LAST_TAG' : 8
}

EXT4_JNL_CHKSUM_TYPE = {
    'JBD2_CRC32_CHKSUM' : 1,
    'JBD2_MD5_CHKSUM' : 2,
    'JBD2_SHA1_CHKSUM' : 3
}

No_of_block_in_a_group = 32768
size_of_each_block_group = 134217728
#
# Ext4 Hash
#
EXT4_HASH_VERSION = {
    'DX_HASH_LEGACY' : 0x0,
    'DX_HASH_HALF_MD4' : 0x1,
    'DX_HASH_TEA' : 0x2,
    'DX_HASH_LEGACY_UNSIGNED' : 0x3,
    'DX_HASH_HALF_MD4_UNSIGNED' : 0x4,
    'DX_HASH_TEA_UNSIGNED' : 0x5
}

def timestamp_to_utc_string(timestamp):
    time_struct = time.gmtime(timestamp)
    return time.strftime('%Y-%m-%d %H:%M:%S', time_struct)

class Ext4Parser:
    def __init__(self, filepath):
        self.console = Console()
        # self.filepath = filepath
        # self.filepath = filepath

        self.filepath = filepath
        file_size = Path(filepath).stat().st_size
        with Progress(
            SpinnerColumn(),
            TextColumn("[bold green]Reading file..."),
            transient=True
        ) as progress:
            task = progress.add_task("[cyan]Loading...", total=file_size)

            with open(filepath, "rb") as f:
                self.f = f.read() # Read the entire file

            # Complete the task once file is fully read
            progress.update(task, advance=file_size)
            console.print("[bold green]File read successfully!/[bold green]")

        # print(f"File {filepath} read successfully. Total size: {len(self.f)} bytes")

    self.ext4_superblock = {
        'sb_inodes_count' : 0,
        'sb_blocks_count_lo' : 0,
        'sb_r_blocks_count_lo' : 0,
        'sb_free_blocks_count_lo' : 0,
        'sb_free_inodes_count' : 0,
        'sb_first_data_block' : 0,
        'sb_log_block_size' : 0,
        'sb_obso_log_frag_size' : 0,
        'sb_blocks_per_group' : 0,
        'sb_obso_frags_per_group' : 0,
        'sb_inodes_per_group' : 0,
        'sb_mtime' : 0,
        'sb_wtime' : 0,
        'sb_mnt_count' : 0,
        'sb_max_mnt_count' : 0xFFFF,
        'sb_magic' : EXT4_SUPER_MAGIC,
        'sb_state' : EXT4_STATE['EXT4_VALID_FS'],
        'sb_errors' : EXT4_ERRORS['EXT4_ERRORS_RO'],
        'sb_minor_rev_level' : 0,

```

```

'sb_lastcheck'      : 0,
'sb_checkinterval'  : 0,
'sb_creator_os'     : EXT4_OS['EXT4_OS_LINUX'],
'sb_rev_level'      : EXT4_REV_LEVEL['EXT4_DYNAMIC_REV'],
'sb_def_resuid'     : EXT4_DEF_RESERVED_ID['EXT4_DEF_RESUID'],
'sb_def_resgid'     : EXT4_DEF_RESERVED_ID['EXT4_DEF_RESUID'],
#if EXT4_DYNAMIC_REV superblocks
'sb_first_ino'      : EXT4_INODE_NO['EXT4_GOOD_OLD_FIRST_INO'],
'sb_inode_size'     : 0,
'sb_block_group_nr' : 0,
'sb_feature_compat' : 0,
'sb_feature_incompat' : 0,
'sb_feature_ro_compat' : 0,
'sb_uuid'           : 0,
'sb_volume_name'    : "",
'sb_last_mounted'   : "",
'sb_algorithm_usage_bitmap' : 0,
'sb_prealloc_blocks' : 0,
'sb_prealloc_dir_blocks' : 0,
'sb_reserved_gdt_blocks' : 0,
# Journaling if EXT4_FEATURE_COMPAT_HAS_JOURNAL
'sb_journal_uuid'   : 0,
'sb_journal_inum'   : EXT4_INODE_NO['EXT4_JOURNAL_INO'],
'sb_journal_dev'    : 0,
'sb_last_orphan'    : 0,
'sb_hash_seed'      : 0,
'sb_def_hash_version' : EXT4_HASH_VERSION['DX_HASH_TEA'],
'sb_reserved_char_pad' : EXT4_JNL_BACKUP_BLOCKS,
'sb_desc_size'      : 0,
'sb_default_mount_opts' : 0x0000,
'sb_first_meta_bg'  : 0,
'sb_mkfs_time'      : 0,
'sb_jnl_blocks'     : 0,
#EXT4_FEATURE_INCOMPAT_64BIT
'sb_blocks_count_hi' : 0,
'sb_r_blocks_count_hi' : 0,
'sb_free_blocks_count_hi' : 0,
'sb_min_extra_isize' : 0,
'sb_want_extra_isize' : 0,
'sb_flags'          : 0x0000,
'sb_raid_stride'    : 0,
'sb_mmp_interval'   : 0,
'sb_mmp_block'      : 0,
'sb_raid_stripe_width' : 0,
'sb_log_groups_per_flex' : 0,
'sb_reserved_char_pad2' : 0,
'sb_reserved_pad'   : 0,
'sb_kbytes_written' : 0,
'sb_reserved'       : 0,
}

self.ext4_blockgroupdescriptor = {
'bg_block_bitmap_lo' : 0,
'bg_inode_bitmap_lo' : 0,
'bg_inode_table_lo' : 0,
'bg_free_blocks_count_lo' : 0,
'bg_free_inodes_count_lo' : 0,
'bg_used_dirs_count_lo' : 0,
'bg_flags' : EXT4_BG_FLAGS['EXT2_BG_INODE_UNINIT'],
'bg_exclude_bitmap_lo' : 0,
'bg_reserved1' : 0,
'bg_itable_unused_lo' : 0,
'bg_checksum' : 0,
#if EXT4_FEATURE_INCOMPAT_64BIT and 'sb_desc_size' > 32
'bg_block_bitmap_hi' : 0,
'bg_inode_bitmap_hi' : 0,
'bg_inode_table_hi' : 0,
'bg_free_blocks_count_hi' : 0,
'bg_free_inodes_count_hi' : 0,
'bg_used_dirs_count_hi' : 0,
'bg_itable_unused_hi' : 0,
'bg_exclude_bitmap_hi' : 0,
'bg_reserved2' : 0,

```

```

        'bg_reserved3'      : 0,
    }

    self.ext4_inode = {
        'i_mode'             : EXT4_INODE_MODE['S_IXOTH'],
        'i_uid'              : 0,
        'i_size_lo'          : 0,
        'i_atime'            : 0,
        'i_ctime'            : 0,
        'i_mtime'            : 0,
        'i_dtime'            : 0,
        'i_gid'              : 0,
        'i_links_count'      : 0,
        'i_blocks_lo'        : 0,
        'i_flags'            : EXT4_INODE_FLAGS['EXT4_SECRM_FL'],
        'i_i_version'        : 0,
        'i_block'            : 0,
        'i_generation'       : 0,
        'i_file_acl_lo'      : 0,
        'i_size_high'        : 0,
        'i_obso_faddr'       : 0,
        'l_i_blocks_high'    : 0,
        'l_i_file_acl_high'  : 0,
        'l_i_uid_high'       : 0,
        'l_i_gid_high'       : 0,
        'l_i_reserved2'      : 0,
        'i_extra_isize'      : 0,
        'i_pad1'             : 0,
        'i_ctime_extra'      : 0,
        'i_mtime_extra'      : 0,
        'i_atime_extra'      : 0,
        'i_crtime'           : 0,
        'i_crtime_extra'     : 0,
        'i_version_hi'       : 0,
        'i_projid'           : 0,
        'i_reserved'         : 0,
    }

    self.ext4_extent_header = {
        'eh_magic'           : EXT4_EXTENT_TREE_MAGIC,
        'eh_entries'         : 0,
        'eh_max'             : 0,
        'eh_depth'           : 0, # if 0, this extent node points to data blocks, otherwise, this extent node points to other
    extent nodes
        'eh_generation'      : 0,
    }

    self.ext4_extent_header_copy = {
        'eh_magic'           : EXT4_EXTENT_TREE_MAGIC,
        'eh_entries'         : 0,
        'eh_max'             : 0,
        'eh_depth'           : 0, # if 0, this extent node points to data blocks, otherwise, this extent node points to other
    extent nodes
        'eh_generation'      : 0,
    }

    self.ext4_extent = {
        'ee_block'           : 0,
        'ee_len'             : 0,
        'ee_start_hi'        : 0,
        'ee_start_lo'        : 0,
    }

    self.ext4_extent_copy = {
        'ee_block'           : 0,
        'ee_len'             : 0,
        'ee_start_hi'        : 0,
        'ee_start_lo'        : 0,
    }

    self.ext4_extent_idx = {
        'ei_block'           : 0,
        'ei_leaf_lo'         : 0,
        'ei_leaf_hi'         : 0,
    }

```

```

        'ei_unused'          : 0,
    }

    self.ext4_extent_idx_copy = {
        'ei_block'           : 0,
        'ei_leaf_lo'         : 0,
        'ei_leaf_hi'         : 0,
        'ei_unused'          : 0,
    }

    self.ext4_dir_entry = {
        'inode'               : 0,
        'rec_len'             : 0,
        'name_len'            : 0,
        'name'                : "",
    }

    self.ext4_dir_entry_2 = {
        'inode'               : 0,
        'rec_len'             : 0,
        'name_len'            : 0,
        'file_type'           : EXT4_FILE_TYPE[EXT4_FT_UNKNOWN],
        'name'                : "",
    }

    self.dx_root = {
        'dot_inode'           : 0,
        'dot_rec_len'         : 0,
        'dot_name_len'        : 0,
        'dot_file_type'       : 0,
        'dot_name'            : "",
        'dot_dot_inode'       : 0,
        'dot_dot_rec_len'     : 0,
        'dot_dot_name_len'    : 0,
        'dot_dot_file_type'   : 0,
        'dot_dot_name'        : "",
        'reserved_zero'       : 0,
        'hash_version'        : EXT4_HASH_VERSION[DX_HASH_LEGACY],
        'info_length'         : 0,
        'indirect_levels'     : 0,
        'unused_flags'        : 0,
        'limit'               : 0,
        'count'               : 0,
        'block'               : 0,
        'entries'             : 0,
    }

    self.dx_node = {
        'fake_inode'          : 0,
        'fake_rec_len'        : 0,
        'fake_name_len'       : 0,
        'fake_file_type'      : 0,
        'limit'               : 0,
        'count'               : 0,
        'block'               : 0,
        'entries'             : 0,
    }

    self.dx_entry = {
        'hash'                : 0,
        'block'               : 0,
    }

    self.ext4_xattr_header = {
        'xh_magic'            : EXT4_XATTR_MAGIC,
        'xh_refcount'         : 0,
        'xh_blocks'           : 0,
        'xh_hash'             : 0,
        'xh_reserved'         : 0,
    }

    self.ext4_xattr_entry = {
        'xe_name_len'         : 0,
        'xe_name_index'       : 0,
    }

```

```

        'xe_value_offs'      : 0,
        'xe_value_block'    : 0,
        'xe_value_size'     : 0,
        'xe_hash'           : 0,
        'xe_name'           : "",
    }

    self.DEBUG = False

    def str2int_le(self, str_list):
        return int.from_bytes(str_list.encode(), byteorder='little')

    def parse_ext4(self):
        offset = 0
        offset = offset + 1024 # Superblock is at 1024 bytes
        self.parse_ext4_superblock(offset)
        # exit()
        offset = offset + 1024 #end of superblock
        offset = offset + 2048 #start of block group descriptor and end of free space
        # count_of_blocks = self.ext4_superblock['sb_blocks_count_lo']
        # bg_num = math.ceil((count_of_blocks - self.ext4_superblock['sb_first_data_block']) /
self.ext4_superblock['sb_blocks_per_group'])
        # print(f"Total Block Groups: {bg_num}")
        count_of_bg = math.ceil(self.ext4_superblock['sb_blocks_count_lo'] / self.ext4_superblock['sb_blocks_per_group'])
        print(f"Total Block Groups: {count_of_bg}")
        # exit()
        og_offset = offset
        for i in range(count_of_bg):
            print(f"\n\nParsing Block Group {i}:\n\n")
            self.parse_ext4_block_group_descriptor(og_offset)
            if self.ext4_superblock['sb_desc_size'] == 32:
                # print("Azr43l")
                # exit()
                og_offset = og_offset + 32
            else:
                og_offset = og_offset + 64
            # if self.ext4_superblock['sb_feature_ro_compat'] &
EXT4_FEATURE_RO_COMPAT[EXT4_FEATURE_RO_COMPAT_SPARSE_SUPER]:
            #     offset = offset + 32 * count_of_bg
            # else:
            #     offset = offset + 64 * count_of_bg
            # print(f"End of Block Group Descriptor Table: {hex(offset)}")
            # print("OFFSET: ", hex(og_offset))
            # exit()
            for i in range(count_of_bg):
                print(f"\n\nParsing Inode Table for Block Group {i}:\n\n")
                current_bg_offset = int.from_bytes(self.f[offset+8:offset+12], byteorder='little')
                inode_table_offset = current_bg_offset * EXT4_BLOCK_SZ
                # print(f"Offset of Inode Table: {hex(inode_table_offset)}")
                # print("offset", hex(offset))
                self.parse_ext4_inode_table(inode_table_offset, i)
                if self.ext4_superblock['sb_desc_size'] == 32:
                    # print("Azr43l")
                    # exit()
                    offset = offset + 32
                else:
                    offset = offset + 64
            # print(f"End of Inode Table: {hex(offset)}")

    def parse_ext4_superblock(self, offset):
        self.ext4_superblock['sb_inodes_count'] = int.from_bytes(self.f[offset+0x00:offset+0x04], byteorder='little')
        print(f"Total Inodes: {self.ext4_superblock['sb_inodes_count']}")
        self.ext4_superblock['sb_blocks_count_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        print(f"Total Blocks: {self.ext4_superblock['sb_blocks_count_lo']}")
        self.ext4_superblock['sb_r_blocks_count_lo'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
        print(f"Reserved Blocks: {self.ext4_superblock['sb_r_blocks_count_lo']}")
        self.ext4_superblock['sb_free_blocks_count_lo'] = int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
        print(f"Free Blocks: {self.ext4_superblock['sb_free_blocks_count_lo']}")
        self.ext4_superblock['sb_free_inodes_count'] = int.from_bytes(self.f[offset+0x10:offset+0x14], byteorder='little')
        print(f"Free Inodes: {self.ext4_superblock['sb_free_inodes_count']}")
        self.ext4_superblock['sb_first_data_block'] = int.from_bytes(self.f[offset+0x14:offset+0x18], byteorder='little')
        print(f"First Data Block: {self.ext4_superblock['sb_first_data_block']}")
        self.ext4_superblock['sb_log_block_size'] = int.from_bytes(self.f[offset+0x18:offset+0x1C], byteorder='little')

```



```

print(f"Log Block Size: {self.ext4_superblock['sb_log_block_size']}")
self.ext4_superblock['sb_obso_log_frag_size'] = int.from_bytes(self.f[offset+0x1C:offset+0x20], byteorder='little')
print(f"Obsolete Log Fragment Size: {self.ext4_superblock['sb_obso_log_frag_size']}")
self.ext4_superblock['sb_blocks_per_group'] = int.from_bytes(self.f[offset+0x20:offset+0x24], byteorder='little')
print(f"Blocks per Group: {self.ext4_superblock['sb_blocks_per_group']}")
self.ext4_superblock['sb_obso_frags_per_group'] = int.from_bytes(self.f[offset+0x24:offset+0x28], byteorder='little')
print(f"Obsolete Fragments per Group: {self.ext4_superblock['sb_obso_frags_per_group']}")
self.ext4_superblock['sb_inodes_per_group'] = int.from_bytes(self.f[offset+0x28:offset+0x2C], byteorder='little')
print(f"Inodes per Group: {self.ext4_superblock['sb_inodes_per_group']}")
self.ext4_superblock['sb_mtime'] = int.from_bytes(self.f[offset+0x2C:offset+0x30], byteorder='little')
print(f"Mount Time: {datetime.utcfromtimestamp(self.ext4_superblock['sb_mtime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_superblock['sb_wtime'] = int.from_bytes(self.f[offset+0x30:offset+0x34], byteorder='little')
print(f"Write Time: {datetime.utcfromtimestamp(self.ext4_superblock['sb_wtime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_superblock['sb_mnt_count'] = int.from_bytes(self.f[offset+0x34:offset+0x36], byteorder='little')
print(f"Mount Count: {self.ext4_superblock['sb_mnt_count']}")
self.ext4_superblock['sb_max_mnt_count'] = int.from_bytes(self.f[offset+0x36:offset+0x38], byteorder='little')
print(f"Max Mount Count: {self.ext4_superblock['sb_max_mnt_count']}")
self.ext4_superblock['sb_magic'] = int.from_bytes(self.f[offset+0x38:offset+0x3A], byteorder='little')
print(f"Magic Number: {self.ext4_superblock['sb_magic']}")
self.ext4_superblock['sb_state'] = int.from_bytes(self.f[offset+0x3A:offset+0x3C], byteorder='little')
state = ""
if self.ext4_superblock['sb_state'] == 1:
    state = "Valid FS"
elif self.ext4_superblock['sb_state'] == 2:
    state = "Error FS"
elif self.ext4_superblock['sb_state'] == 4:
    state = "Orphan FS"
print(f"State: {state}")
self.ext4_superblock['sb_errors'] = int.from_bytes(self.f[offset+0x3C:offset+0x3E], byteorder='little')
errors = ""
if self.ext4_superblock['sb_errors'] == 1:
    errors = "Continue"
elif self.ext4_superblock['sb_errors'] == 2:
    errors = "Read-Only"
elif self.ext4_superblock['sb_errors'] == 3:
    errors = "Panic"
print(f"Errors: {errors}")
self.ext4_superblock['sb_minor_rev_level'] = int.from_bytes(self.f[offset+0x3E:offset+0x40], byteorder='little')
print(f"Minor Revision Level: {self.ext4_superblock['sb_minor_rev_level']}")
self.ext4_superblock['sb_lastcheck'] = int.from_bytes(self.f[offset+0x40:offset+0x44], byteorder='little')
print(f"Last Check: {datetime.utcfromtimestamp(self.ext4_superblock['sb_lastcheck']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_superblock['sb_checkinterval'] = int.from_bytes(self.f[offset+0x44:offset+0x48], byteorder='little')
print(f"Check Interval: {self.ext4_superblock['sb_checkinterval']}")
self.ext4_superblock['sb_creator_os'] = int.from_bytes(self.f[offset+0x48:offset+0x4C], byteorder='little')
creator_os = ""
if self.ext4_superblock['sb_creator_os'] == 0:
    creator_os = "Linux"
elif self.ext4_superblock['sb_creator_os'] == 1:
    creator_os = "Hurd"
elif self.ext4_superblock['sb_creator_os'] == 2:
    creator_os = "Masix"
elif self.ext4_superblock['sb_creator_os'] == 3:
    creator_os = "FreeBSD"
elif self.ext4_superblock['sb_creator_os'] == 4:
    creator_os = "Lites"
print(f"Creator OS: {creator_os}")
self.ext4_superblock['sb_rev_level'] = int.from_bytes(self.f[offset+0x4C:offset+0x50], byteorder='little')
rev_level = ""
if self.ext4_superblock['sb_rev_level'] == 0:
    rev_level = "Good Old Rev"
elif self.ext4_superblock['sb_rev_level'] == 1:
    rev_level = "Dynamic Rev"
print(f"Revision Level: {rev_level}")
self.ext4_superblock['sb_def_resuid'] = int.from_bytes(self.f[offset+0x50:offset+0x52], byteorder='little')
print(f"Default Reserved UID: {self.ext4_superblock['sb_def_resuid']}")
self.ext4_superblock['sb_def_resgid'] = int.from_bytes(self.f[offset+0x52:offset+0x54], byteorder='little')
print(f"Default Reserved GID: {self.ext4_superblock['sb_def_resgid']}")
print(f"\n\nFor EXT4_DYNAMIC_REV Super Blocks Only\n\n")
self.ext4_superblock['sb_first_ino'] = int.from_bytes(self.f[offset+0x54:offset+0x58], byteorder='little')
print(f"First Inode: {self.ext4_superblock['sb_first_ino']}")
self.ext4_superblock['sb_inode_size'] = int.from_bytes(self.f[offset+0x58:offset+0x5A], byteorder='little')
print(f"Inode Size: {self.ext4_superblock['sb_inode_size']}")
self.ext4_superblock['sb_block_group_nr'] = int.from_bytes(self.f[offset+0x5A:offset+0x5C], byteorder='little')

```

```

print(f"Block Group Number: {self.ext4_superblock['sb_block_group_nr']}")
self.ext4_superblock['sb_feature_compat'] = int.from_bytes(self.f[offset+0x5C:offset+0x60], byteorder='little')
feature_compat = ""
if self.ext4_superblock['sb_feature_compat'] & 0x0001:
    feature_compat += " Directory Preallocation "
if self.ext4_superblock['sb_feature_compat'] & 0x0002:
    feature_compat += " Imagic Inodes "
if self.ext4_superblock['sb_feature_compat'] & 0x0004:
    feature_compat += " Has Journal "
if self.ext4_superblock['sb_feature_compat'] & 0x0008:
    feature_compat += " Extended Attributes "
if self.ext4_superblock['sb_feature_compat'] & 0x0010:
    feature_compat += " Resize Inode "
if self.ext4_superblock['sb_feature_compat'] & 0x0020:
    feature_compat += " Directory Index "
print("Compatible feature: " + feature_compat)
self.ext4_superblock['sb_feature_incompat'] = int.from_bytes(self.f[offset+0x60:offset+0x64], byteorder='little')
feature_incompat = ""
if self.ext4_superblock['sb_feature_incompat'] & 0x0001:
    feature_incompat += " Compression "
if self.ext4_superblock['sb_feature_incompat'] & 0x0002:
    feature_incompat += " Filetype "
if self.ext4_superblock['sb_feature_incompat'] & 0x0004:
    feature_incompat += " Recover "
if self.ext4_superblock['sb_feature_incompat'] & 0x0008:
    feature_incompat += " Journal_Device "
if self.ext4_superblock['sb_feature_incompat'] & 0x0010:
    feature_incompat += " Meta_Block_Group "
if self.ext4_superblock['sb_feature_incompat'] & 0x0040:
    feature_incompat += " Extents "
if self.ext4_superblock['sb_feature_incompat'] & 0x0080:
    feature_incompat += " 64-bit "
if self.ext4_superblock['sb_feature_incompat'] & 0x0100:
    feature_incompat += " MMP "
if self.ext4_superblock['sb_feature_incompat'] & 0x0200:
    feature_incompat += " Flex_Block_Group "
if self.ext4_superblock['sb_feature_incompat'] & 0x0400:
    feature_incompat += " EA_Inode "
if self.ext4_superblock['sb_feature_incompat'] & 0x1000:
    feature_incompat += " Directory_Data "
if self.ext4_superblock['sb_feature_incompat'] & 0x2000:
    feature_incompat += " Reserved "
if self.ext4_superblock['sb_feature_incompat'] & 0x4000:
    feature_incompat += " Snapshot "
if self.ext4_superblock['sb_feature_incompat'] & 0x8000:
    feature_incompat += " Snapshot_Quota "
if self.ext4_superblock['sb_feature_incompat'] & 0x10000:
    feature_incompat += " Encrypted "
if self.ext4_superblock['sb_feature_incompat'] & 0x20000:
    feature_incompat += " Casefold "
print("Incompatible feature: " + feature_incompat)
self.ext4_superblock['sb_feature_ro_compat'] = int.from_bytes(self.f[offset+0x64:offset+0x68], byteorder='little')
feature_ro_compat = ""
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0001:
    feature_ro_compat += " Sparse_Super "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0002:
    feature_ro_compat += " Large_File "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0004:
    feature_ro_compat += " Btree_Directory "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0008:
    feature_ro_compat += " Huge_File "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0010:
    feature_ro_compat += " GDT_CSUM "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0020:
    feature_ro_compat += " Directory_NLink "
if self.ext4_superblock['sb_feature_ro_compat'] & 0x0040:
    feature_ro_compat += " Extra_1Size "
print("Read-Only Compatible feature: " + feature_ro_compat)
self.ext4_superblock['sb_uuid'] = self.f[offset+0x68:offset+0x78].hex()
formatted_uuid = ''.join(self.ext4_superblock['sb_uuid'][i:i+2] for i in range(0, len(self.ext4_superblock['sb_uuid']), 2))
print(f"UUID: {formatted_uuid}")
self.ext4_superblock['sb_volume_name'] = self.f[offset+0x78:offset+0x88].decode('utf-8').rstrip("\x00")
print(f"Volume Name: {self.ext4_superblock['sb_volume_name']}")
self.ext4_superblock['sb_last_mounted'] = self.f[offset+0x88:offset+0xc8].decode('utf-8').rstrip("\x00")

```

```

print(f"Last Mounted: {self.ext4_superblock['sb_last_mounted']}")
self.ext4_superblock['sb_algorithm_usage_bitmap'] = int.from_bytes(self.f[offset+0xC8:offset+0xCC], byteorder='little')
print(f"Algorithm Usage Bitmap: {self.ext4_superblock['sb_algorithm_usage_bitmap']}")
self.ext4_superblock['sb_prealloc_blocks'] = int.from_bytes(self.f[offset+0xCC:offset+0xCD], byteorder='little')
print(f"\n\nEXT4_FEATURE_COMPAT_DIR_PREALLOC on for performance hints Directory preallocation\n\n")
print(f"Preallocated Blocks: {self.ext4_superblock['sb_prealloc_blocks']}")
self.ext4_superblock['sb_prealloc_dir_blocks'] = int.from_bytes(self.f[offset+0xCD:offset+0xCE], byteorder='little')
print(f"Preallocated Directory Blocks: {self.ext4_superblock['sb_prealloc_dir_blocks']}")
self.ext4_superblock['sb_reserved_gdt_blocks'] = int.from_bytes(self.f[offset+0xCE:offset+0xD0], byteorder='little')
print(f"Reserved GDT Blocks: {self.ext4_superblock['sb_reserved_gdt_blocks']}")
print(f"\n\nFor EXT4_FEATURE_COMPAT_HAS_JOURNAL Only\n\n")
self.ext4_superblock['sb_journal_uuid'] = self.f[offset+0xD0:offset+0xE0].hex()
formatted_journal_uuid = ' '.join(self.ext4_superblock['sb_journal_uuid'][i:i+2] for i in range(0,
len(self.ext4_superblock['sb_journal_uuid']), 2))
print(f"Journal UUID: {formatted_journal_uuid}")
self.ext4_superblock['sb_journal_inum'] = int.from_bytes(self.f[offset+0xE0:offset+0xE4], byteorder='little')
print(f"Journal Inode: {self.ext4_superblock['sb_journal_inum']}")
self.ext4_superblock['sb_journal_dev'] = int.from_bytes(self.f[offset+0xE4:offset+0xE8], byteorder='little')
print(f"Journal Device: {self.ext4_superblock['sb_journal_dev']}")
self.ext4_superblock['sb_last_orphan'] = int.from_bytes(self.f[offset+0xE8:offset+0xEC], byteorder='little')
print(f"Last Orphan: {self.ext4_superblock['sb_last_orphan']}")
self.ext4_superblock['sb_hash_seed'] = int.from_bytes(self.f[offset+0xEC:offset+0xFC], byteorder='little')
print(f"HTREE Hash Seed: {self.ext4_superblock['sb_hash_seed']}")
self.ext4_superblock['sb_def_hash_version'] = int.from_bytes(self.f[offset+0xFC:offset+0xFD], byteorder='little')
def_hashversion = ""
if self.ext4_superblock['sb_def_hash_version'] == 0:
    def_hashversion = "Legacy"
elif self.ext4_superblock['sb_def_hash_version'] == 1:
    def_hashversion = "Half MD4"
elif self.ext4_superblock['sb_def_hash_version'] == 2:
    def_hashversion = "TEA"
elif self.ext4_superblock['sb_def_hash_version'] == 3:
    def_hashversion = "Legacy Unsigned"
elif self.ext4_superblock['sb_def_hash_version'] == 4:
    def_hashversion = "Half MD4 Unsigned"
elif self.ext4_superblock['sb_def_hash_version'] == 5:
    def_hashversion = "TEA Unsigned"
print(f"Default Hash Version: {def_hashversion}")
self.ext4_superblock['sb_reserved_char_pad'] = int.from_bytes(self.f[offset+0xFD:offset+0xFE], byteorder='little')
print(f"Reserved Char Pad: {self.ext4_superblock['sb_reserved_char_pad']}")
self.ext4_superblock['sb_desc_size'] = int.from_bytes(self.f[offset+0xFE:offset+0x100], byteorder='little')
print(f"Descriptor Size: {self.ext4_superblock['sb_desc_size']}")
self.ext4_superblock['sb_default_mount_opts'] = int.from_bytes(self.f[offset+0x100:offset+0x104], byteorder='little')
default_mount_opts = ""
if self.ext4_superblock['sb_default_mount_opts'] & 0x0001:
    default_mount_opts += " Debug "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0002:
    default_mount_opts += " BSD Groups "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0004:
    default_mount_opts += " XATTR User "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0008:
    default_mount_opts += " ACL "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0010:
    default_mount_opts += " UID16 "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0020:
    default_mount_opts += " JMODE Data "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0040:
    default_mount_opts += " JMODE Ordered "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0060:
    default_mount_opts += " JMODE "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0100:
    default_mount_opts += " No Barrier "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0200:
    default_mount_opts += " Block Validity "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0400:
    default_mount_opts += " Discard "
if self.ext4_superblock['sb_default_mount_opts'] & 0x0800:
    default_mount_opts += " No Delayed Allocation "
print(f"Default Mount Options: {default_mount_opts}")
self.ext4_superblock['sb_first_meta_bg'] = int.from_bytes(self.f[offset+0x104:offset+0x108], byteorder='little')
print(f"First Meta Block Group: {self.ext4_superblock['sb_first_meta_bg']}")
self.ext4_superblock['sb_mkfs_time'] = int.from_bytes(self.f[offset+0x108:offset+0x10C], byteorder='little')
print(f"MKFS Time: {datetime.datetime.fromtimestamp(self.ext4_superblock['sb_mkfs_time']).strftime('%Y-%m-%d
%H:%M:%S')}")

```

```

self.ext4_superblock['sb_jnl_blocks'] = int.from_bytes(self.f[offset+0x108:offset+0x150], byteorder='little')
# print(f"Journal Backup: {self.ext4_superblock['sb_jnl_blocks']}")
self.ext4_superblock['sb_blocks_count_hi'] = int.from_bytes(self.f[offset+0x150:offset+0x154], byteorder='little')
self.ext4_superblock['sb_r_blocks_count_hi'] = int.from_bytes(self.f[offset+0x154:offset+0x158], byteorder='little')
self.ext4_superblock['sb_free_blocks_count_hi'] = int.from_bytes(self.f[offset+0x158:offset+0x15C], byteorder='little')
self.ext4_superblock['sb_min_extra_isize'] = int.from_bytes(self.f[offset+0x15C:offset+0x15E], byteorder='little')
self.ext4_superblock['sb_want_extra_isize'] = int.from_bytes(self.f[offset+0x15E:offset+0x160], byteorder='little')
self.ext4_superblock['sb_flags'] = int.from_bytes(self.f[offset+0x160:offset+0x164], byteorder='little')
miscflags = ""
if self.ext4_superblock['sb_flags'] & 0x0001:
    miscflags += " Signed_Directory_Hash "
if self.ext4_superblock['sb_flags'] & 0x0002:
    miscflags += " Unsigned_Directory_Hash "
if self.ext4_superblock['sb_flags'] & 0x0004:
    miscflags += " Test_File_System "
if self.ext4_superblock['sb_flags'] & 0x0010:
    miscflags += " Snapshot_Enabled "
if self.ext4_superblock['sb_flags'] & 0x0020:
    miscflags += " Snapshot_Fix "
if self.ext4_superblock['sb_flags'] & 0x0040:
    miscflags += " Fix_Exclude "
self.ext4_superblock['sb_raid_stride'] = int.from_bytes(self.f[offset+0x164:offset+0x166], byteorder='little')
self.ext4_superblock['sb_mmp_interval'] = int.from_bytes(self.f[offset+0x166:offset+0x168], byteorder='little')
self.ext4_superblock['sb_mmp_block'] = int.from_bytes(self.f[offset+0x168:offset+0x170], byteorder='little')
self.ext4_superblock['sb_raid_stripe_width'] = int.from_bytes(self.f[offset+0x170:offset+0x174], byteorder='little')
self.ext4_superblock['sb_log_groups_per_flex'] = int.from_bytes(self.f[offset+0x174:offset+0x175], byteorder='little')
# print(f"Log Groups per Flex: {self.ext4_superblock['sb_log_groups_per_flex']}")
self.ext4_superblock['sb_reserved_char_pad2'] = int.from_bytes(self.f[offset+0x175:offset+0x176], byteorder='little')
self.ext4_superblock['sb_reserved_pad'] = self.f[offset+0x176:offset+0x178].hex()
self.ext4_superblock['sb_kbytes_written'] = int.from_bytes(self.f[offset+0x178:offset+0x180], byteorder='little')
self.ext4_superblock['sb_reserved'] = self.f[offset+0x180:offset+0x400].hex()
print("\n\nEnd of Superblock Parsing\n\n")

def parse_ext4_block_group_descriptor(self, offset):
    self.ext4_blockgroupdescriptor['bg_block_bitmap_lo'] = int.from_bytes(self.f[offset+0x00:offset+0x04], byteorder='little')
    print(f"Block Bitmap: {self.ext4_blockgroupdescriptor['bg_block_bitmap_lo']}")
    self.ext4_blockgroupdescriptor['bg_inode_bitmap_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
    print(f"Inode Bitmap: {self.ext4_blockgroupdescriptor['bg_inode_bitmap_lo']}")
    self.ext4_blockgroupdescriptor['bg_inode_table_lo'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Inode Table: {self.ext4_blockgroupdescriptor['bg_inode_table_lo']}")
    self.ext4_blockgroupdescriptor['bg_free_blocks_count_lo'] = int.from_bytes(self.f[offset+0x0C:offset+0x0E],
byteorder='little')
    print(f"Free Blocks: {self.ext4_blockgroupdescriptor['bg_free_blocks_count_lo']}")
    self.ext4_blockgroupdescriptor['bg_free_inodes_count_lo'] = int.from_bytes(self.f[offset+0x0E:offset+0x10],
byteorder='little')
    print(f"Free Inodes: {self.ext4_blockgroupdescriptor['bg_free_inodes_count_lo']}")
    self.ext4_blockgroupdescriptor['bg_used_dirs_count_lo'] = int.from_bytes(self.f[offset+0x10:offset+0x12],
byteorder='little')
    print(f"Used Directories: {self.ext4_blockgroupdescriptor['bg_used_dirs_count_lo']}")
    self.ext4_blockgroupdescriptor['bg_flags'] = int.from_bytes(self.f[offset+0x12:offset+0x14], byteorder='little')
    flags = ""
    if self.ext4_blockgroupdescriptor['bg_flags'] == 0:
        flags = "Inode Uninit"
    elif self.ext4_blockgroupdescriptor['bg_flags'] == 1:
        flags = "Block Uninit"
    print(f"Flags: {flags}")
    self.ext4_blockgroupdescriptor['bg_exclude_bitmap_lo'] = int.from_bytes(self.f[offset+0x14:offset+0x18],
byteorder='little')
    print(f"Exclude Bitmap: {self.ext4_blockgroupdescriptor['bg_exclude_bitmap_lo']}")
    self.ext4_blockgroupdescriptor['bg_reserved1'] = int.from_bytes(self.f[offset+0x18:offset+0x1C], byteorder='little')
    print(f"Reserved1: {self.ext4_blockgroupdescriptor['bg_reserved1']}")
    self.ext4_blockgroupdescriptor['bg_itable_unused_lo'] = int.from_bytes(self.f[offset+0x1C:offset+0x1E],
byteorder='little')
    print(f"Unused Inode Table: {self.ext4_blockgroupdescriptor['bg_itable_unused_lo']}")
    self.ext4_blockgroupdescriptor['bg_checksum'] = int.from_bytes(self.f[offset+0x1E:offset+0x20], byteorder='little')
    print(f"Checksum: {self.ext4_blockgroupdescriptor['bg_checksum']}")
    if self.ext4_superblock['sb_feature_incompat'] & EXT4_FEATURE_INCOMPAT[EXT4_FEATURE_INCOMPAT_64BIT]:
    and self.ext4_superblock['sb_desc_size'] > 32:
        print("\n\nFor EXT4_FEATURE_INCOMPAT_64BIT and 'sb_desc_size' > 32\n\n")
        self.ext4_blockgroupdescriptor['bg_block_bitmap_hi'] = int.from_bytes(self.f[offset+0x20:offset+0x24],
byteorder='little')
        print(f"Block Bitmap Hi: {self.ext4_blockgroupdescriptor['bg_block_bitmap_hi']}")
        self.ext4_blockgroupdescriptor['bg_inode_bitmap_hi'] = int.from_bytes(self.f[offset+0x24:offset+0x28],

```

```

byteorder='little')
    print(f"Node Bitmap Hi: {self.ext4_blockgroupdescriptor['bg_inode_bitmap_hi']}")
    self.ext4_blockgroupdescriptor['bg_inode_table_hi'] = int.from_bytes(self.f[offset+0x28:offset+0x2C],
byteorder='little')
    print(f"Node Table Hi: {self.ext4_blockgroupdescriptor['bg_inode_table_hi']}")
    self.ext4_blockgroupdescriptor['bg_free_blocks_count_hi'] = int.from_bytes(self.f[offset+0x2C:offset+0x2E],
byteorder='little')
    print(f"Free Blocks Hi: {self.ext4_blockgroupdescriptor['bg_free_blocks_count_hi']}")
    self.ext4_blockgroupdescriptor['bg_free_inodes_count_hi'] = int.from_bytes(self.f[offset+0x2E:offset+0x30],
byteorder='little')
    print(f"Free Inodes Hi: {self.ext4_blockgroupdescriptor['bg_free_inodes_count_hi']}")
    self.ext4_blockgroupdescriptor['bg_used_dirs_count_hi'] = int.from_bytes(self.f[offset+0x30:offset+0x32],
byteorder='little')
    print(f"Used Directories Hi: {self.ext4_blockgroupdescriptor['bg_used_dirs_count_hi']}")
    self.ext4_blockgroupdescriptor['bg_itable_unused_hi'] = int.from_bytes(self.f[offset+0x32:offset+0x34],
byteorder='little')
    print(f"Unused Inode Table Hi: {self.ext4_blockgroupdescriptor['bg_itable_unused_hi']}")
    self.ext4_blockgroupdescriptor['bg_exclude_bitmap_hi'] = int.from_bytes(self.f[offset+0x34:offset+0x38],
byteorder='little')
    print(f"Exclude Bitmap Hi: {self.ext4_blockgroupdescriptor['bg_exclude_bitmap_hi']}")
    self.ext4_blockgroupdescriptor['bg_reserved2'] = int.from_bytes(self.f[offset+0x38:offset+0x3C], byteorder='little')
    print(f"Reserved2: {self.ext4_blockgroupdescriptor['bg_reserved2']}")
    self.ext4_blockgroupdescriptor['bg_reserved3'] = int.from_bytes(self.f[offset+0x3C:offset+0x40], byteorder='little')
    print(f"Reserved3: {self.ext4_blockgroupdescriptor['bg_reserved3']}")

def parse_ext4_inode_table(self, offset, group_num):
    inode_size = self.ext4_superblock['sb_inode_size']
    inode_count = self.ext4_superblock['sb_inodes_per_group']
    for i in range(inode_count):
        # print(f"\n\nParsing Inode {i}:\n\n")
        # if i==525749:
        #     exit()
        self.parse_ext4_inode(offset+(i*inode_size), i, group_num)
        inodeoffset=offset+(i*inode_size)
        if int.from_bytes(self.f[inodeoffset+0x04:inodeoffset+0x08], byteorder='little') == 0:
            continue
        if int.from_bytes(self.f[inodeoffset+0x02:inodeoffset+0x04], byteorder='little') == 0 and
int.from_bytes(self.f[inodeoffset+0x28:inodeoffset+0x2C], byteorder='little') == 0:
            continue
        print("\n-----Parsing Extended Attributes-----\n")
        self.ext4_parse_xattr((offset+(i*inode_size))+160)
        print("\n-----End of Extended Attributes-----\n")
        print("\n-----Parsing Extent Tree-----\n")
        self.parse_ext4_extenttree(offset+(i*inode_size)+0x28)
        print("\n-----End of Extent Tree-----\n")
        extoffset=offset+((i*inode_size)+0x28)
        if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0:
            self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02], byteorder='little')
            self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04], byteorder='little')
            self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
            self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08], byteorder='little')
            self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
            if self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==0:
                ee_block=[]
                save_extoffset=extoffset
                for i in range(self.ext4_extent_header_copy['eh_entries']):
                    self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                    self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
                    self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
                    self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
                    ee_block.append(self.ext4_extent['ee_block'])
                    extoffset=extoffset+12
                log_offset=[]
                log_number=[]
                extoffset=save_extoffset
                for i in range(self.ext4_extent_header_copy['eh_entries']):
                    ogloglen=len(log_number)
                    self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                    self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
                    self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
                    self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
                    for i in range(len(log_number), ee_block[i]):
                        log_number.append(i)

```

```

        log_offset.append(self.ext4_extent['ee_start_lo']*(i-ogloglen)*4096)
        extoffset=extoffset+0x12
        extoffset=save_extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            self.ext4_parse_hashtree(self.ext4_extent['ee_start_lo'],log_number,log_offset)
            extoffset=extoffset+0x18
        elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==1:
            self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
            self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
            extoffset=extoffset+0x18
            extoffset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
            self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
            self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
            self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
            self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
            self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
            ee_block=[]
            save_extoffset=extoffset
            for i in range(self.ext4_extent_header_copy['eh_entries']):
                self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
                self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
                self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
                ee_block.append(self.ext4_extent['ee_block'])
                extoffset=extoffset+12
                log_offset=[]
                log_number=[]
                extoffset=save_extoffset
            for i in range(self.ext4_extent_header_copy['eh_entries']):
                ogloglen=len(log_number)
                self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
                self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
                self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
                for i in range(len(log_number),ee_block[i]):
                    log_number.append(i)
                    log_offset.append(self.ext4_extent['ee_start_lo']*(i-ogloglen)*4096)
                extoffset=extoffset+0x12
            extoffset=save_extoffset
            for i in range(self.ext4_extent_header_copy['eh_entries']):
                self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
                self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
                self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
                self.ext4_parse_hashtree(self.ext4_extent['ee_start_lo'],log_number,log_offset)
                extoffset=extoffset+0x18
        elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==2:
            self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
            self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
            extoffset=extoffset+0x18
            extoffset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
            self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
            self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
            self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
            self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
            self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
            self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')

```

```

        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
        extoffset=extoffset+0x18
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
        ee_block=[]
        save_extoffset=extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            ee_block.append(self.ext4_extent['ee_block'])
            extoffset=extoffset+12
            log_offset=[]
            log_number=[]
            extoffset=save_extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            ogloglen=len(log_number)
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            for i in range(len(log_number),ee_block[i]):
                log_number.append(i)
                log_offset.append(self.ext4_extent['ee_start_lo']*(i-ogloglen)*4096)
            extoffset=extoffset+0x12
        extoffset=save_extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            self.ext4_parse_hashtree(self.ext4_extent['ee_start_lo'],log_number,log_offset)
            extoffset=extoffset+0x18
    elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==3:
        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
        extoffset=extoffset+0x18
        self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
        extoffset=extoffset+0x18
        self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')

```

```

        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
        extoffset=extoffset+0x18
        extoffset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
        ee_block=[]
        save_extoffset=extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            ee_block.append(self.ext4_extent['ee_block'])
            extoffset=extoffset+12
            log_offset=[]
            log_number=[]
            extoffset=save_extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            ogloglen=len(log_number)
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            for i in range(len(log_number),ee_block[i]):
                log_number.append(i)
                log_offset.append(self.ext4_extent['ee_start_lo']*(i-ogloglen)*4096)
            extoffset=extoffset+0x12
        extoffset=save_extoffset
        for i in range(self.ext4_extent_header_copy['eh_entries']):
            self.ext4_extent['ee_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent['ee_len'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x12], byteorder='little')
            self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[extoffset+0x12:extoffset+0x14], byteorder='little')
            self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x18], byteorder='little')
            self.ext4_parse_hashtree(self.ext4_extent['ee_start_lo'],log_number,log_offset)
            extoffset=extoffset+0x18
        # print("Block: {self.ext4_extent['ee_block']} Length: {self.ext4_extent['ee_len']} Start:
{self.ext4_extent['ee_start_hi']}:{self.ext4_extent['ee_start_lo']}")
        else:
            self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[extoffset+0x00:extoffset+0x02], byteorder='little')
            self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[extoffset+0x02:extoffset+0x04], byteorder='little')
            self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[extoffset+0x04:extoffset+0x06], byteorder='little')
            self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[extoffset+0x06:extoffset+0x08], byteorder='little')
            self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[extoffset+0x08:extoffset+0x0C],
byteorder='little')
            if self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==0:
                self.ext4_parse_direntry(extoffset+0x0C)
                # if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS[EXT4_INDEX_FL] != 0:
                #     continue
            elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==1:
                self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
                self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
                self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
                self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
                extoffset=extoffset+0x18
                new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
                self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
                self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
                self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
                self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
                self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],

```



```

byteorder='little')
    new_offset=new_offset+0x0C
    self.ext4_parse_dirent(new_offset)
    # if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0:
    #     continue
    elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==2:
        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
        extoffset=extoffset+0x18
        new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],
byteorder='little')
        new_offset=new_offset+0x0C
        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[new_offset+0x0C:new_offset+0x10],
byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[new_offset+0x10:new_offset+0x14],
byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[new_offset+0x14:new_offset+0x16],
byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[new_offset+0x16:new_offset+0x18],
byteorder='little')
        new_offset=new_offset+0x18
        new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],
byteorder='little')
        new_offset=new_offset+0x0C
        self.ext4_parse_dirent(new_offset)
        # if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0:
        #     continue
        elif self.ext4_extent_header_copy['eh_magic']==0xF30A and self.ext4_extent_header_copy['eh_depth']==3:
            self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[extoffset+0x0C:extoffset+0x10], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[extoffset+0x10:extoffset+0x14], byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[extoffset+0x14:extoffset+0x16], byteorder='little')
            self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[extoffset+0x16:extoffset+0x18], byteorder='little')
            extoffset=extoffset+0x18
            new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
            self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
            self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
            self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
            self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
            self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],
byteorder='little')
            new_offset=new_offset+0x0C
            self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[new_offset+0x0C:new_offset+0x10],
byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[new_offset+0x10:new_offset+0x14],
byteorder='little')
            self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[new_offset+0x14:new_offset+0x16],
byteorder='little')
            self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[new_offset+0x16:new_offset+0x18],
byteorder='little')
            new_offset=new_offset+0x18

```

```

        new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],
byteorder='little')
        new_offset=new_offset+0x0C
        self.ext4_extent_idx_copy['ei_block'] = int.from_bytes(self.f[new_offset+0x0C:new_offset+0x10],
byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_lo'] = int.from_bytes(self.f[new_offset+0x10:new_offset+0x14],
byteorder='little')
        self.ext4_extent_idx_copy['ei_leaf_hi'] = int.from_bytes(self.f[new_offset+0x14:new_offset+0x16],
byteorder='little')
        self.ext4_extent_idx_copy['ei_unused'] = int.from_bytes(self.f[new_offset+0x16:new_offset+0x18],
byteorder='little')
        new_offset=new_offset+0x18
        new_offset=self.ext4_extent_idx_copy['ei_leaf_lo']*4096
        self.ext4_extent_header_copy['eh_magic'] = int.from_bytes(self.f[new_offset+0x00:new_offset+0x02],
byteorder='little')
        self.ext4_extent_header_copy['eh_entries'] = int.from_bytes(self.f[new_offset+0x02:new_offset+0x04],
byteorder='little')
        self.ext4_extent_header_copy['eh_max'] = int.from_bytes(self.f[new_offset+0x04:new_offset+0x06],
byteorder='little')
        self.ext4_extent_header_copy['eh_depth'] = int.from_bytes(self.f[new_offset+0x06:new_offset+0x08],
byteorder='little')
        self.ext4_extent_header_copy['eh_generation'] = int.from_bytes(self.f[new_offset+0x08:new_offset+0x0C],
byteorder='little')
        new_offset=new_offset+0x0C
        self.ext4_parse_direntry(new_offset)
        # if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0:
        #     continue

    # self.ext4_parse_dir(offset+(i*inode_size))

```

```

def ext4_parse_hashtree(self, offset, log_number, log_offset):
    offset=offset*4096
    self.dx_root['dot_inode']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
    print(f"Inode: {self.dx_root['dot_inode']}")
    self.dx_root['dot_rec_len']=int.from_bytes(self.f[offset+0x04:offset+0x06],byteorder='little')
    print(f"Record Length: {self.dx_root['dot_rec_len']}")
    rec_len=self.dx_root['dot_rec_len']
    self.dx_root['dot_name_len']=int.from_bytes(self.f[offset+0x06:offset+0x07],byteorder='little')
    print(f"Name Length: {self.dx_root['dot_name_len']}")
    self.dx_root['dot_file_type']=int.from_bytes(self.f[offset+0x07:offset+0x08],byteorder='little')
    print(f"File Type: {self.dx_root['dot_file_type']}")
    self.dx_root['dot_name']=self.f[offset+0x08:offset+0x0C].hex()
    print(f"Name: {self.dx_root['dot_name']}")
    self.dx_root['dot_dot_inode']=int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
    print(f"Inode: {self.dx_root['dot_dot_inode']}")
    self.dx_root['dot_dot_rec_len']=int.from_bytes(self.f[offset+0x10:offset+0x12],byteorder='little')
    print(f"Record Length: {self.dx_root['dot_dot_rec_len']}")
    rec_len=self.dx_root['dot_dot_rec_len']
    self.dx_root['dot_dot_name_len']=int.from_bytes(self.f[offset+0x12:offset+0x13],byteorder='little')
    print(f"Name Length: {self.dx_root['dot_dot_name_len']}")
    self.dx_root['dot_dot_file_type']=int.from_bytes(self.f[offset+0x13:offset+0x14],byteorder='little')
    print(f"File Type: {self.dx_root['dot_dot_file_type']}")
    self.dx_root['dot_dot_name']=self.f[offset+0x14:offset+0x18].hex()
    print(f"Name: {self.dx_root['dot_dot_name']}")
    self.dx_root['reserved_zero']=int.from_bytes(self.f[offset+0x18:offset+0x1C], byteorder='little')
    print(f"Reserved Zero: {self.dx_root['reserved_zero']}")
    self.dx_root['hash_version']=int.from_bytes(self.f[offset+0x1C:offset+0x1D], byteorder='little')
    # print(f"Hash Version: {self.dx_root['hash_version']}")
    hash_version=""
    if self.dx_root['hash_version'] == 0:
        hash_version="Legacy"
    elif self.dx_root['hash_version'] == 1:
        hash_version="Half MD4"
    elif self.dx_root['hash_version'] == 2:

```

```

        hash_version="Tea"
    elif self.dx_root['hash_version'] == 3:
        hash_version="Legacy Unsigned"
    elif self.dx_root['hash_version'] == 4:
        hash_version="Unsigned,Half MD4"
    elif self.dx_root['hash_version'] == 5:
        hash_version="Unsigned,Tea"
    elif self.dx_root['hash_version'] == 6:
        hash_version="Splash"
    print(f"Hash Version: {hash_version}")
    self.dx_root['info_length']=int.from_bytes(self.f[offset+0x1D:offset+0x1E], byteorder='little')
    print(f"Info Length: {self.dx_root['info_length']}")
    self.dx_root['indirect_levels']=int.from_bytes(self.f[offset+0x1E:offset+0x1F], byteorder='little')
    print(f"Indirect Levels: {self.dx_root['indirect_levels']}")
    self.dx_root['unused_flags']=int.from_bytes(self.f[offset+0x1F:offset+0x20], byteorder='little')
    print(f"Unused Flags: {self.dx_root['unused_flags']}")
    self.dx_root['limit']=int.from_bytes(self.f[offset+0x20:offset+0x22], byteorder='little')
    print(f"Limit: {self.dx_root['limit']}")
    self.dx_root['count']=int.from_bytes(self.f[offset+0x22:offset+0x24], byteorder='little')
    print(f"Count: {self.dx_root['count']}")
    self.dx_root['block']=int.from_bytes(self.f[offset+0x24:offset+0x28], byteorder='little')
    print(f"Block: {self.dx_root['block']}")
    offset=offset+0x28
    valid_ent=self.dx_root['count']
    indirectlevel=self.dx_root['indirect_levels']
    if self.dx_root['indirect_levels']==0:
        for i in range(0, valid_ent, 1):
            print(f"Entry: {i}")
            self.dx_entry['hash']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
            print(f"Hash: {self.dx_entry['hash']}")
            self.dx_entry['block']=int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
            print(f"Block: {self.dx_entry['block']}")
            diroff=0
            for i in range(len(log_number)):
                if log_number[i]==self.dx_root['block']:
                    diroff=log_offset[i]
                    break
            offset=offset+0x08
            self.ext4_parse_linear_dir_entry_info(diroff)
    elif self.dx_root['indirect_levels']==1:
        for i in range(0, valid_ent, 1):
            print(f"Entry: {i}")
            self.dx_entry['hash']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
            print(f"Hash: {self.dx_entry['hash']}")
            self.dx_entry['block']=int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
            print(f"Block: {self.dx_entry['block']}")
            diroff=0
            for i in range(len(log_number)):
                if log_number[i]==self.dx_root['block']:
                    diroff=log_offset[i]
                    self.print_ext4_htree(diroff)
                    diroff=diroff+0x08
            for i in range(0, self.dx_root['count'], 1):

                self.dx_entry['hash']=int.from_bytes(self.f[diroff:diroff+0x04], byteorder='little')
                print(f"Hash: {self.dx_entry['hash']}")
                self.dx_entry['block']=int.from_bytes(self.f[diroff+0x04:diroff+0x08], byteorder='little')
                print(f"Block: {self.dx_entry['block']}")
                dir2=0
                for i in range(len(log_number)):
                    if log_number[i]==self.dx_entry['block']:
                        dir2=log_offset[i]
                        break
                self.ext4_parse_linear_dir_entry_info(dir2)
                diroff=diroff+0x08
            offset=offset+0x08
    elif self.dx_root['indirect_levels']==2:
        for i in range(0, valid_ent, 1):
            print(f"Entry: {i}")
            self.dx_entry['hash']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
            print(f"Hash: {self.dx_entry['hash']}")
            self.dx_entry['block']=int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
            print(f"Block: {self.dx_entry['block']}")
            diroff=0

```

```

for i in range(len(log_number)):
    if log_number[i]==self.dx_root['block']:
        diroff=log_offset[i]
        self.print_ext4_htree(diroff)
        diroff=diroff+0x08
    for i in range(0, self.dx_root['count'], 1):
        self.dx_entry['hash']=int.from_bytes(self.f[diroff:diroff+0x04], byteorder='little')
        print(f"Hash: {self.dx_entry['hash']}")
        self.dx_entry['block']=int.from_bytes(self.f[diroff+0x04:diroff+0x08], byteorder='little')
        print(f"Block: {self.dx_entry['block']}")
        diroff2=0
    for i in range(len(log_number)):
        if log_number[i]==self.dx_entry['block']:
            diroff2=log_offset[i]
            self.print_ext4_htree(diroff2)
            diroff2=diroff2+0x08
        for i in range(0, self.dx_root['count'], 1):
            self.dx_entry['hash']=int.from_bytes(self.f[diroff2:diroff2+0x04], byteorder='little')
            print(f"Hash: {self.dx_entry['hash']}")
            self.dx_entry['block']=int.from_bytes(self.f[diroff2+0x04:diroff2+0x08], byteorder='little')
            print(f"Block: {self.dx_entry['block']}")
            dir3=0
        for i in range(len(log_number)):
            if log_number[i]==self.dx_entry['block']:
                dir3=log_offset[i]
                break
        self.ext4_parse_linear_dir_entry_info(dir3)
        diroff2=diroff2+0x08
    diroff=diroff+0x08
    offset=offset+0x08
elif self.dx_root['indirect_levels']==3:
    for i in range(0, valid_ent, 1):
        print(f"Entry: {i}")
        self.dx_entry['hash']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        print(f"Hash: {self.dx_entry['hash']}")
        self.dx_entry['block']=int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        print(f"Block: {self.dx_entry['block']}")
        diroff=0
    for i in range(len(log_number)):
        if log_number[i]==self.dx_root['block']:
            diroff=log_offset[i]
            self.print_ext4_htree(diroff)
            diroff=diroff+0x08
        for i in range(0, self.dx_root['count'], 1):
            self.dx_entry['hash']=int.from_bytes(self.f[diroff:diroff+0x04], byteorder='little')
            print(f"Hash: {self.dx_entry['hash']}")
            self.dx_entry['block']=int.from_bytes(self.f[diroff+0x04:diroff+0x08], byteorder='little')
            print(f"Block: {self.dx_entry['block']}")
            diroff2=0
        for i in range(len(log_number)):
            if log_number[i]==self.dx_entry['block']:
                diroff2=log_offset[i]
                self.print_ext4_htree(diroff2)
                diroff2=diroff2+0x08
            for i in range(0, self.dx_root['count'], 1):
                self.dx_entry['hash']=int.from_bytes(self.f[diroff2:diroff2+0x04], byteorder='little')
                print(f"Hash: {self.dx_entry['hash']}")
                self.dx_entry['block']=int.from_bytes(self.f[diroff2+0x04:diroff2+0x08], byteorder='little')
                print(f"Block: {self.dx_entry['block']}")
                diroff3=0
            for i in range(len(log_number)):
                if log_number[i]==self.dx_entry['block']:
                    diroff3=log_offset[i]
                    self.print_ext4_htree(diroff3)
                    diroff3=diroff3+0x08
                for i in range(0, self.dx_root['count'], 1):
                    self.dx_entry['hash']=int.from_bytes(self.f[diroff3:diroff3+0x04], byteorder='little')
                    print(f"Hash: {self.dx_entry['hash']}")
                    self.dx_entry['block']=int.from_bytes(self.f[diroff3+0x04:diroff3+0x08], byteorder='little')
                    print(f"Block: {self.dx_entry['block']}")
                    dir4=0
                for i in range(len(log_number)):
                    if log_number[i]==self.dx_entry['block']:
                        dir4=log_offset[i]

```

```

        break
        self.ext4_parse_linear_dir_entry_info(dir4)
        diroff3=diroff3+0x08
        diroff2=diroff2+0x08
        diroff=diroff+0x08
        offset=offset+0x08

def print_ext4_htree(self,offset):
    self.dx_root['dot_inode']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
    print(f"inode: {self.dx_root['dot_inode']}")
    self.dx_root['dot_rec_len']=int.from_bytes(self.f[offset+0x04:offset+0x06],byteorder='little')
    print(f"Record Length: {self.dx_root['dot_rec_len']}")
    rec_len=self.dx_root['dot_rec_len']
    self.dx_root['dot_name_len']=int.from_bytes(self.f[offset+0x06:offset+0x07],byteorder='little')
    print(f"Name Length: {self.dx_root['dot_name_len']}")
    self.dx_root['dot_file_type']=int.from_bytes(self.f[offset+0x07:offset+0x08],byteorder='little')
    print(f"File Type: {self.dx_root['dot_file_type']}")
    self.dx_root['dot_name']=self.f[offset+0x08:offset+0x0C].hex()
    print(f"Name: {self.dx_root['dot_name']}")
    self.dx_root['dot_dot_inode']=int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
    print(f"inode: {self.dx_root['dot_dot_inode']}")
    self.dx_root['dot_dot_rec_len']=int.from_bytes(self.f[offset+0x10:offset+0x12],byteorder='little')
    print(f"Record Length: {self.dx_root['dot_dot_rec_len']}")
    rec_len=self.dx_root['dot_dot_rec_len']
    self.dx_root['dot_dot_name_len']=int.from_bytes(self.f[offset+0x12:offset+0x13],byteorder='little')
    print(f"Name Length: {self.dx_root['dot_dot_name_len']}")
    self.dx_root['dot_dot_file_type']=int.from_bytes(self.f[offset+0x13:offset+0x14],byteorder='little')
    print(f"File Type: {self.dx_root['dot_dot_file_type']}")
    self.dx_root['dot_dot_name']=self.f[offset+0x14:offset+0x18].hex()
    print(f"Name: {self.dx_root['dot_dot_name']}")
    self.dx_root['reserved_zero']=int.from_bytes(self.f[offset+0x18:offset+0x1C], byteorder='little')
    print(f"Reserved Zero: {self.dx_root['reserved_zero']}")
    self.dx_root['hash_version']=int.from_bytes(self.f[offset+0x1C:offset+0x1D], byteorder='little')
    # print(f"Hash Version: {self.dx_root['hash_version']}")
    hash_version=""
    if self.dx_root['hash_version'] == 0:
        hash_version="Legacy"
    elif self.dx_root['hash_version'] == 1:
        hash_version="Half MD4"
    elif self.dx_root['hash_version'] == 2:
        hash_version="Tea"
    elif self.dx_root['hash_version'] == 3:
        hash_version="Legacy Unsigned"
    elif self.dx_root['hash_version'] == 4:
        hash_version="Unsigned,Half MD4"
    elif self.dx_root['hash_version'] == 5:
        hash_version="Unsigned,Tea"
    elif self.dx_root['hash_version'] == 6:
        hash_version="Splash"
    print(f"Hash Version: {hash_version}")
    self.dx_root['info_length']=int.from_bytes(self.f[offset+0x1D:offset+0x1E], byteorder='little')
    print(f"Info Length: {self.dx_root['info_length']}")
    self.dx_root['indirect_levels']=int.from_bytes(self.f[offset+0x1E:offset+0x1F], byteorder='little')
    print(f"Indirect Levels: {self.dx_root['indirect_levels']}")
    self.dx_root['unused_flags']=int.from_bytes(self.f[offset+0x1F:offset+0x20], byteorder='little')
    print(f"Unused Flags: {self.dx_root['unused_flags']}")
    self.dx_root['limit']=int.from_bytes(self.f[offset+0x20:offset+0x22], byteorder='little')
    print(f"Limit: {self.dx_root['limit']}")
    self.dx_root['count']=int.from_bytes(self.f[offset+0x22:offset+0x24], byteorder='little')
    print(f"Count: {self.dx_root['count']}")
    self.dx_root['block']=int.from_bytes(self.f[offset+0x24:offset+0x28], byteorder='little')
    print(f"Block: {self.dx_root['block']}")

def parse_ext4_inode(self,offset,inode_num,group_num):
    if self.DEBUG==False and int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little') == 0:
        return
    if int.from_bytes(self.f[offset+0x02:offset+0x04],byteorder='little')==0 and
int.from_bytes(self.f[offset+0x28:offset+0x2C],byteorder='little')==0:
        return
    print(f"\n\nParsing Inode {(group_num*self.ext4_superblock['sb_inodes_per_group']+inode_num+1):\n\n")
    idchk=((group_num*self.ext4_superblock['sb_inodes_per_group']+inode_num+1)
    # print(hex(offset))
    # if idchk == 525749:

```

```

# exit()
# print(inode_num)
# print(group_num)
# print(self.ext4_superblock['sb_inodes_per_group'])
self.ext4_inode['i_mode'] = int.from_bytes(self.f[offset+0x00:offset+0x02], byteorder='little')
print(f"Mode: {self.ext4_inode['i_mode']}")
self.ext4_inode['i_uid'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
print(f"UID: {self.ext4_inode['i_uid']}")
self.ext4_inode['i_size_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
print(f"Size: {self.ext4_inode['i_size_lo']}")
self.ext4_inode['i_atime'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
print(f"Access Time: {datetime.utcfromtimestamp(self.ext4_inode['i_atime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_inode['i_ctime'] = int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
print(f"Creation Time: {datetime.utcfromtimestamp(self.ext4_inode['i_ctime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_inode['i_mtime'] = int.from_bytes(self.f[offset+0x10:offset+0x14], byteorder='little')
print(f"Modification Time: {datetime.utcfromtimestamp(self.ext4_inode['i_mtime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_inode['i_dtime'] = int.from_bytes(self.f[offset+0x14:offset+0x18], byteorder='little')
print(f"Deletion Time: {datetime.utcfromtimestamp(self.ext4_inode['i_dtime']).strftime('%Y-%m-%d %H:%M:%S')}")
self.ext4_inode['i_gid'] = int.from_bytes(self.f[offset+0x18:offset+0x1A], byteorder='little')
print(f"GID: {self.ext4_inode['i_gid']}")
self.ext4_inode['i_links_count'] = int.from_bytes(self.f[offset+0x1A:offset+0x1C], byteorder='little')
print(f"Links Count: {self.ext4_inode['i_links_count']}")
self.ext4_inode['i_blocks_lo'] = int.from_bytes(self.f[offset+0x1C:offset+0x20], byteorder='little')
print(f"Blocks: {self.ext4_inode['i_blocks_lo']}")
self.ext4_inode['i_flags'] = int.from_bytes(self.f[offset+0x20:offset+0x24], byteorder='little')
print(f"Flags: {self.ext4_inode['i_flags']}")
self.ext4_inode['i_osd1'] = self.f[offset+0x24:offset+0x28].hex()
print(f"OSD1: {self.ext4_inode['i_osd1']}")
self.ext4_inode['i_block'] = []
for i in range(15):
    self.ext4_inode['i_block'].append(int.from_bytes(self.f[offset+0x28+(i*4):offset+0x2C+(i*4)], byteorder='little'))
print(f"Blocks: {self.ext4_inode['i_block']}")
self.ext4_inode['i_generation'] = int.from_bytes(self.f[offset+0x64:offset+0x68], byteorder='little')
print(f"Generation: {self.ext4_inode['i_generation']}")
self.ext4_inode['i_file_acl_lo'] = int.from_bytes(self.f[offset+0x68:offset+0x6C], byteorder='little')
print(f"File ACL: {self.ext4_inode['i_file_acl_lo']}")
self.ext4_inode['i_size_high'] = int.from_bytes(self.f[offset+0x6C:offset+0x70], byteorder='little')
print(f"Size High: {self.ext4_inode['i_size_high']}")
self.ext4_inode['i_obso_faddr'] = int.from_bytes(self.f[offset+0x70:offset+0x74], byteorder='little')
print(f"Obsolete Fragment Address: {self.ext4_inode['i_obso_faddr']}")
self.ext4_inode['i_i_blocks_high'] = int.from_bytes(self.f[offset+0x74:offset+0x78], byteorder='little')
print(f"Blocks High: {self.ext4_inode['i_i_blocks_high']}")
self.ext4_inode['i_i_file_acl_high'] = int.from_bytes(self.f[offset+0x78:offset+0x7C], byteorder='little')
print(f"File ACL High: {self.ext4_inode['i_i_file_acl_high']}")
self.ext4_inode['i_i_uid_high'] = int.from_bytes(self.f[offset+0x7C:offset+0x80], byteorder='little')
print(f"UID High: {self.ext4_inode['i_i_uid_high']}")
self.ext4_inode['i_i_gid_high'] = int.from_bytes(self.f[offset+0x80:offset+0x84], byteorder='little')
print(f"GID High: {self.ext4_inode['i_i_gid_high']}")
self.ext4_inode['i_i_checksum_lo'] = int.from_bytes(self.f[offset+0x84:offset+0x88], byteorder='little')
print(f"Checksum: {self.ext4_inode['i_i_checksum_lo']}")
self.ext4_inode['i_i_reserved'] = int.from_bytes(self.f[offset+0x88:offset+0x90], byteorder='little')
print(f"Reserved: {self.ext4_inode['i_i_reserved']}")
self.ext4_inode['i_i_checksum_hi'] = int.from_bytes(self.f[offset+0x90:offset+0x94], byteorder='little')
print(f"Checksum High: {self.ext4_inode['i_i_checksum_hi']}")
self.ext4_inode['i_i_extra_isize'] = int.from_bytes(self.f[offset+0x94:offset+0x96], byteorder='little')
print(f"Extra ISize: {self.ext4_inode['i_i_extra_isize']}")
self.ext4_inode['i_i_ctime_extra'] = int.from_bytes(self.f[offset+0x96:offset+0x98], byteorder='little')
print(f"CTime Extra: {self.ext4_inode['i_i_ctime_extra']}")
self.ext4_inode['i_i_mtime_extra'] = int.from_bytes(self.f[offset+0x98:offset+0x9A], byteorder='little')
print(f"MTIME Extra: {self.ext4_inode['i_i_mtime_extra']}")
self.ext4_inode['i_i_atime_extra'] = int.from_bytes(self.f[offset+0x9A:offset+0x9C], byteorder='little')
print(f"ATime Extra: {self.ext4_inode['i_i_atime_extra']}")
self.ext4_inode['i_i_crtime'] = int.from_bytes(self.f[offset+0x9C:offset+0xA0], byteorder='little')
print(f"CRTime: {self.ext4_inode['i_i_crtime']}")
self.ext4_inode['i_i_crtime_extra'] = int.from_bytes(self.f[offset+0xA0:offset+0xA2], byteorder='little')
print(f"CRTime Extra: {self.ext4_inode['i_i_crtime_extra']}")
self.ext4_inode['i_i_version_hi'] = int.from_bytes(self.f[offset+0xA2:offset+0xA4], byteorder='little')
print(f"Version High: {self.ext4_inode['i_i_version_hi']}")
self.ext4_inode['i_i_projid'] = int.from_bytes(self.f[offset+0xA4:offset+0xA6], byteorder='little')
print(f"Project ID: {self.ext4_inode['i_i_projid']}")
self.ext4_inode['i_i_reserved2'] = int.from_bytes(self.f[offset+0xA6:offset+0x100], byteorder='little')
print("\n")

```

```
def parse_ext4_extentree(self, offset):
```

```

self.ext4_extent_header['eh_magic'] = int.from_bytes(self.f[offset+0x00:offset+0x02], byteorder='little')
print(f"Magic: {self.ext4_extent_header['eh_magic']}")
self.ext4_extent_header['eh_entries'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
print(f"Entries: {self.ext4_extent_header['eh_entries']}")
self.ext4_extent_header['eh_max'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
print(f"Max: {self.ext4_extent_header['eh_max']}")
self.ext4_extent_header['eh_depth'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
print(f"Depth: {self.ext4_extent_header['eh_depth']}")
self.ext4_extent_header['eh_generation'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
print(f"Generation: {self.ext4_extent_header['eh_generation']}")
offset=offset+0x0C
for i in range(0, self.ext4_extent_header['eh_entries'], 1):
    if self.ext4_extent_header['eh_depth'] == 0:
        self.ext4_extent['ee_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent['ee_len'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
        self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
        self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
        print(f"\n-----Parsing ext4 extent-----\n")
        print(f"Block: {self.ext4_extent['ee_block']}")
        print(f"Length: {self.ext4_extent['ee_len']}")
        print(f"Start Hi: {self.ext4_extent['ee_start_hi']}")
        print(f"Start Lo: {self.ext4_extent['ee_start_lo']}")
        offset=offset+0x0C
    elif self.ext4_extent_header['eh_depth'] == 1:
        self.ext4_extent_idx['ei_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent_idx['ei_leaf_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        self.ext4_extent_idx['ei_leaf_hi'] = int.from_bytes(self.f[offset+0x08:offset+0x0A], byteorder='little')
        self.ext4_extent_idx['ei_unused'] = int.from_bytes(self.f[offset+0x0A:offset+0x0C], byteorder='little')
        print(f"\n-----Parsing 1D ext4 extent index-----\n")
        print(f"Block: {self.ext4_extent_idx['ei_block']}")
        print(f"Leaf Lo: {self.ext4_extent_idx['ei_leaf_lo']}")
        print(f"Leaf Hi: {self.ext4_extent_idx['ei_leaf_hi']}")
        print(f"Unused: {self.ext4_extent_idx['ei_unused']}")
        offset=offset+0x0C
        self.ext4_parse_extenttree_idx1_pointer(self.ext4_extent_idx['ei_leaf_lo']*4096)
    elif self.ext4_extent_header['eh_depth'] == 2:
        self.ext4_extent_idx['ei_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent_idx['ei_leaf_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        self.ext4_extent_idx['ei_leaf_hi'] = int.from_bytes(self.f[offset+0x08:offset+0x0A], byteorder='little')
        self.ext4_extent_idx['ei_unused'] = int.from_bytes(self.f[offset+0x0A:offset+0x0C], byteorder='little')
        print(f"\n-----Parsing 2D ext4 extent index-----\n")
        print(f"Block: {self.ext4_extent_idx['ei_block']}")
        print(f"Leaf Lo: {self.ext4_extent_idx['ei_leaf_lo']}")
        print(f"Leaf Hi: {self.ext4_extent_idx['ei_leaf_hi']}")
        print(f"Unused: {self.ext4_extent_idx['ei_unused']}")
        offset=offset+0x0C
        self.ext4_parse_extenttree_idx2_pointer(self.ext4_extent_idx['ei_leaf_lo']*4096)
    elif self.ext4_extent_header['eh_depth'] == 3:
        self.ext4_extent_idx['ei_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent_idx['ei_leaf_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        self.ext4_extent_idx['ei_leaf_hi'] = int.from_bytes(self.f[offset+0x08:offset+0x0A], byteorder='little')
        self.ext4_extent_idx['ei_unused'] = int.from_bytes(self.f[offset+0x0A:offset+0x0C], byteorder='little')
        print(f"\n-----Parsing 3D ext4 extent index-----\n")
        print(f"Block: {self.ext4_extent_idx['ei_block']}")
        print(f"Leaf Lo: {self.ext4_extent_idx['ei_leaf_lo']}")
        print(f"Leaf Hi: {self.ext4_extent_idx['ei_leaf_hi']}")
        print(f"Unused: {self.ext4_extent_idx['ei_unused']}")
        offset=offset+0x0C
        self.ext4_parse_extenttree_idx3_pointer(self.ext4_extent_idx['ei_leaf_lo']*4096)

def ext4_parse_extenttree_idx1_pointer(self, offset):
    self.ext4_extent_header['eh_magic'] = int.from_bytes(self.f[offset+0x00:offset+0x02], byteorder='little')
    print(f"Magic: {self.ext4_extent_header['eh_magic']}")
    self.ext4_extent_header['eh_entries'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
    print(f"Entries: {self.ext4_extent_header['eh_entries']}")
    self.ext4_extent_header['eh_max'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
    print(f"Max: {self.ext4_extent_header['eh_max']}")
    self.ext4_extent_header['eh_depth'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
    print(f"Depth: {self.ext4_extent_header['eh_depth']}")
    self.ext4_extent_header['eh_generation'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Generation: {self.ext4_extent_header['eh_generation']}")
    offset=offset+0x0C

```

```

for i in range(0, self.ext4_extent_header['eh_entries'], 1):
    self.ext4_extent['ee_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
    self.ext4_extent['ee_len'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
    self.ext4_extent['ee_start_hi'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
    self.ext4_extent['ee_start_lo'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print("\n-----Parsing 1D ext4 extent-----\n")
    print(f"Block: {self.ext4_extent['ee_block']}")
    print(f"Length: {self.ext4_extent['ee_len']}")
    print(f"Start Hi: {self.ext4_extent['ee_start_hi']}")
    print(f"Start Lo: {self.ext4_extent['ee_start_lo']}")
    offset=offset+0x0C

def ext4_parse_extenttree_idx2_pointer(self,offset):
    self.ext4_extent_header['eh_magic'] = int.from_bytes(self.f[offset+0x00:offset+0x02], byteorder='little')
    print(f"Magic: {self.ext4_extent_header['eh_magic']}")
    self.ext4_extent_header['eh_entries'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
    print(f"Entries: {self.ext4_extent_header['eh_entries']}")
    self.ext4_extent_header['eh_max'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
    print(f"Max: {self.ext4_extent_header['eh_max']}")
    self.ext4_extent_header['eh_depth'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
    print(f"Depth: {self.ext4_extent_header['eh_depth']}")
    self.ext4_extent_header['eh_generation'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Generation: {self.ext4_extent_header['eh_generation']}")
    offset=offset+0x0C
    for i in range(0, self.ext4_extent_header['eh_entries'], 1):
        self.ext4_extent_idx['ei_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent_idx['ei_leaf_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        self.ext4_extent_idx['ei_leaf_hi'] = int.from_bytes(self.f[offset+0x08:offset+0x0A], byteorder='little')
        self.ext4_extent_idx['ei_unused'] = int.from_bytes(self.f[offset+0x0A:offset+0x0C], byteorder='little')
        print("\n-----Parsing 2D ext4 extent index-----\n")
        print(f"Block: {self.ext4_extent_idx['ei_block']}")
        print(f"Leaf Lo: {self.ext4_extent_idx['ei_leaf_lo']}")
        print(f"Leaf Hi: {self.ext4_extent_idx['ei_leaf_hi']}")
        print(f"Unused: {self.ext4_extent_idx['ei_unused']}")
        offset=offset+0x0C
        self.ext4_parse_extenttree_idx1_pointer(self.ext4_extent_idx['ei_leaf_lo']*4096)

def ext4_parse_extenttree_idx3_pointer(self,offset):
    self.ext4_extent_header['eh_magic'] = int.from_bytes(self.f[offset+0x00:offset+0x02], byteorder='little')
    print(f"Magic: {self.ext4_extent_header['eh_magic']}")
    self.ext4_extent_header['eh_entries'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
    print(f"Entries: {self.ext4_extent_header['eh_entries']}")
    self.ext4_extent_header['eh_max'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
    print(f"Max: {self.ext4_extent_header['eh_max']}")
    self.ext4_extent_header['eh_depth'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
    print(f"Depth: {self.ext4_extent_header['eh_depth']}")
    self.ext4_extent_header['eh_generation'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Generation: {self.ext4_extent_header['eh_generation']}")
    offset=offset+0x0C
    for i in range(0, self.ext4_extent_header['eh_entries'], 1):
        self.ext4_extent_idx['ei_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        self.ext4_extent_idx['ei_leaf_lo'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
        self.ext4_extent_idx['ei_leaf_hi'] = int.from_bytes(self.f[offset+0x08:offset+0x0A], byteorder='little')
        self.ext4_extent_idx['ei_unused'] = int.from_bytes(self.f[offset+0x0A:offset+0x0C], byteorder='little')
        print("\n-----Parsing 3D ext4 extent index-----\n")
        print(f"Block: {self.ext4_extent_idx['ei_block']}")
        print(f"Leaf Lo: {self.ext4_extent_idx['ei_leaf_lo']}")
        print(f"Leaf Hi: {self.ext4_extent_idx['ei_leaf_hi']}")
        print(f"Unused: {self.ext4_extent_idx['ei_unused']}")
        offset=offset+0x0C
        self.ext4_parse_extenttree_idx2_pointer(self.ext4_extent_idx['ei_leaf_lo']*4096)

def ext4_parse_xattr(self,offset):
    self.ext4_xattr_header['xh_magic'] = int.from_bytes(self.f[offset+0x00:offset+0x04], byteorder='little')
    print(f"Magic: {self.ext4_xattr_header['xh_magic']}")
    self.ext4_xattr_header['xh_refcount'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
    print(f"Refcount: {self.ext4_xattr_header['xh_refcount']}")
    self.ext4_xattr_header['xh_blocks'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Blocks: {self.ext4_xattr_header['xh_blocks']}")
    self.ext4_xattr_header['xh_hash'] = int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
    print(f"Hash: {self.ext4_xattr_header['xh_hash']}")
    self.ext4_xattr_header['xh_reserved'] = []
    for i in range(4):
        self.ext4_xattr_header['xh_reserved'].append(int.from_bytes(self.f[offset+0x10+(i*4):offset+0x14+(i*4)],

```



```

byteorder='little'))
    print(f"Reserved: {self.ext4_xattr_header['xh_reserved']}")
    offset=offset+0x20
    offset=offset+16
    self.ext4_xattr_entry['xe_name_entry'] = int.from_bytes(self.f[offset:offset+0x01], byteorder='little')
    print(f"Name Entry: {self.ext4_xattr_entry['xe_name_entry']}")
    self.ext4_xattr_entry['xe_name_index'] = int.from_bytes(self.f[offset+0x01:offset+0x02], byteorder='little')
    print(f"Name Index: {self.ext4_xattr_entry['xe_name_index']}")
    self.ext4_xattr_entry['xe_value_offs'] = int.from_bytes(self.f[offset+0x02:offset+0x04], byteorder='little')
    print(f"Value Offset: {self.ext4_xattr_entry['xe_value_offs']}")
    self.ext4_xattr_entry['xe_value_block'] = int.from_bytes(self.f[offset+0x04:offset+0x08], byteorder='little')
    print(f"Value Block: {self.ext4_xattr_entry['xe_value_block']}")
    self.ext4_xattr_entry['xe_value_size'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    print(f"Value Size: {self.ext4_xattr_entry['xe_value_size']}")
    self.ext4_xattr_entry['xe_hash'] = int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
    print(f"Hash: {self.ext4_xattr_entry['xe_hash']}")
    self.ext4_xattr_entry['xe_name'] = int.from_bytes(self.f[offset+0x10:offset+0x30], byteorder='little')
    try:
        name = self.f[offset+0x10:offset+0x30].decode('utf-8')
        print(f"Name: {name}")
    except:
        name = self.f[offset+0x10:offset+0x30].hex()
        print(f"Name: {name}")

def ext4_parse_direntry(self,inodeoffset):
    offset=inodeoffset
    self.ext4_extent_copy['ee_block'] = int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
    self.ext4_extent_copy['ee_len'] = int.from_bytes(self.f[offset+0x04:offset+0x06], byteorder='little')
    self.ext4_extent_copy['ee_start_hi'] = int.from_bytes(self.f[offset+0x06:offset+0x08], byteorder='little')
    self.ext4_extent_copy['ee_start_lo'] = int.from_bytes(self.f[offset+0x08:offset+0x0C], byteorder='little')
    offset=self.ext4_extent_copy['ee_start_lo']*4096
    dir_sz = self.ext4_extent_copy['ee_len'] * 4096
    if dir_sz==0:
        return
    i=0
    while i < dir_sz:
        dirent_sz=self.ext4_parse_direntry_internal(offset)
        if dirent_sz==0:
            break
        if (self.ext4_inode['i_mode'] & 0xF000) == 0x4000:
            if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0:
                if self.dx_root['dot_inode'] != 0:
                    pass
                    # print("-----hashtree-----")
                    # exit()
            else:
                if self.ext4_dir_entry_2['inode'] != 0:
                    offset=self.ext4_parse_linear_dir_entry_info(offset)
        i=i+dirent_sz

def ext4_parse_linear_dir_entry_info(self,offset):
    self.ext4_dir_entry_2['inode']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
    print(f"Inode: {self.ext4_dir_entry_2['inode']}")
    self.ext4_dir_entry_2['rec_len']=int.from_bytes(self.f[offset+0x04:offset+0x06],byteorder='little')
    print(f"Record Length: {self.ext4_dir_entry_2['rec_len']}")
    self.ext4_dir_entry_2['name_len']=int.from_bytes(self.f[offset+0x06:offset+0x07],byteorder='little')
    print(f"Name Length: {self.ext4_dir_entry_2['name_len']}")
    self.ext4_dir_entry_2['file_type']=int.from_bytes(self.f[offset+0x07:offset+0x08],byteorder='little')
    print(f"File Type: {self.ext4_dir_entry_2['file_type']}")
    try:
        name = self.f[offset+0x08:offset+0x08+self.ext4_dir_entry_2['name_len']].decode('utf-8')
        print(f"Name: {name}")
    except:
        name = self.f[offset+0x08:offset+0x08+self.ext4_dir_entry_2['name_len']].hex()
        print(f"Name: {name}")
    return (offset+self.ext4_dir_entry_2['rec_len'])

def ext4_parse_direntry_internal(self,offset):
    rec_len=0
    if self.ext4_inode['i_flags'] & EXT4_INODE_FLAGS['EXT4_INDEX_FL'] != 0: #check for hashed entries
        self.dx_root['dot_inode']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
        # print(f"Inode: {self.dx_root['dot_inode']}")

```

```

self.dx_root['dot_rec_len']=int.from_bytes(self.f[offset+0x04:offset+0x06],byteorder='little')
# print(f"Record Length: {self.dx_root['dot_rec_len']}")
rec_len=self.dx_root['dot_rec_len']
self.dx_root['dot_name_len']=int.from_bytes(self.f[offset+0x06:offset+0x07],byteorder='little')
# print(f"Name Length: {self.dx_root['dot_name_len']}")
self.dx_root['dot_file_type']=int.from_bytes(self.f[offset+0x07:offset+0x08],byteorder='little')
# print(f"File Type: {self.dx_root['dot_file_type']}")
self.dx_root['dot_name']=self.f[offset+0x08:offset+0x0C].hex()
# print(f"Name: {self.dx_root['dot_name']}")
self.dx_root['dot_dot_inode']=int.from_bytes(self.f[offset+0x0C:offset+0x10], byteorder='little')
# print(f"Inode: {self.dx_root['dot_dot_inode']}")
self.dx_root['dot_dot_rec_len']=int.from_bytes(self.f[offset+0x10:offset+0x12],byteorder='little')
# print(f"Record Length: {self.dx_root['dot_dot_rec_len']}")
rec_len=self.dx_root['dot_dot_rec_len']
self.dx_root['dot_dot_name_len']=int.from_bytes(self.f[offset+0x12:offset+0x13],byteorder='little')
# print(f"Name Length: {self.dx_root['dot_dot_name_len']}")
self.dx_root['dot_dot_file_type']=int.from_bytes(self.f[offset+0x13:offset+0x14],byteorder='little')
# print(f"File Type: {self.dx_root['dot_dot_file_type']}")
self.dx_root['dot_dot_name']=self.f[offset+0x14:offset+0x18].hex()
# print(f"Name: {self.dx_root['dot_dot_name']}")
self.dx_root['reserved_zero']=int.from_bytes(self.f[offset+0x18:offset+0x1C], byteorder='little')
# print(f"Reserved Zero: {self.dx_root['reserved_zero']}")
self.dx_root['hash_version']=int.from_bytes(self.f[offset+0x1C:offset+0x1D], byteorder='little')
# print(f"Hash Version: {self.dx_root['hash_version']}")
self.dx_root['info_length']=int.from_bytes(self.f[offset+0x1D:offset+0x1E], byteorder='little')
# print(f"Info Length: {self.dx_root['info_length']}")
self.dx_root['indirect_levels']=int.from_bytes(self.f[offset+0x1E:offset+0x1F], byteorder='little')
# print(f"Indirect Levels: {self.dx_root['indirect_levels']}")
self.dx_root['unused_flags']=int.from_bytes(self.f[offset+0x1F:offset+0x20], byteorder='little')
# print(f"Unused Flags: {self.dx_root['unused_flags']}")
self.dx_root['limit']=int.from_bytes(self.f[offset+0x20:offset+0x22], byteorder='little')
# print(f"Limit: {self.dx_root['limit']}")
self.dx_root['count']=int.from_bytes(self.f[offset+0x22:offset+0x24], byteorder='little')
# print(f"Count: {self.dx_root['count']}")
self.dx_root['block']=int.from_bytes(self.f[offset+0x24:offset+0x28], byteorder='little')
# print(f"Block: {self.dx_root['block']}")
return self.dx_root['count']

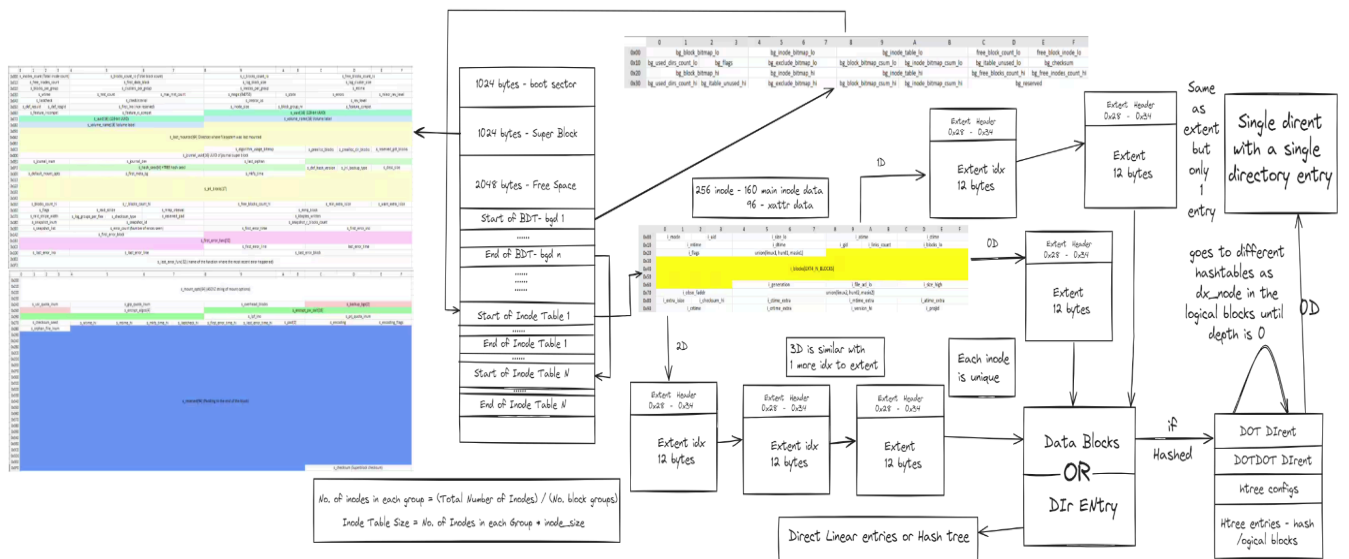
else:
self.ext4_dir_entry_2['inode']=int.from_bytes(self.f[offset:offset+0x04], byteorder='little')
# print(f"Inode: {self.ext4_dir_entry_2['inode']}")
self.ext4_dir_entry_2['rec_len']=int.from_bytes(self.f[offset+0x04:offset+0x06],byteorder='little')
# print(f"Record Length: {self.ext4_dir_entry_2['rec_len']}")
rec_len=self.ext4_dir_entry_2['rec_len']
self.ext4_dir_entry_2['name_len']=int.from_bytes(self.f[offset+0x06:offset+0x07],byteorder='little')
# print(f"Name Length: {self.ext4_dir_entry_2['name_len']}")
self.ext4_dir_entry_2['file_type']=int.from_bytes(self.f[offset+0x07:offset+0x08],byteorder='little')
# print(f"File Type: {self.ext4_dir_entry_2['file_type']}")
offset=offset+0x08
return rec_len

def ext4parser(filepath):
    ext4 = Ext4Parser(filepath)
    ext4.parse_ext4()

console = Console()
banner()
argparse = ArgumentParser(description=__doc__)
argparse.add_argument("extpart", metavar="EXT4 partition")
args = argparse.parse_args()
filename = args.extpart
filepath = Path.cwd() / filename
if filepath.exists():
    console.print(f"\n[bold cyan]Start of Parsing...[/bold cyan]\n")
else:
    print(f"\nFile '{filepath}' not found. Please check the file path.\n")
finalstatus = ext4parser(filepath)

```

Later on we also added hashtree parsing and extended the diagram to make it easier for understanding



Let's look into the parsed superblock

Total Inodes: 1430800
 Total Blocks: 5712123
 Reserved Blocks: 4096
 Free Blocks: 4275127
 Free Inodes: 1416559
 First Data Block: 0
 Log Block Size: 2
 Obsolete Log Fragment Size: 2
 Blocks per Group: 32768
 Obsolete Fragments per Group: 32768
 Inodes per Group: 8176
 Mount Time: 2024-05-05 19:11:54
 Write Time: 2024-05-05 19:11:54
 Mount Count: 3
 Max Mount Count: 65535
 Magic Number: 61267
 State: Valid FS
 Errors: Continue
 Minor Revision Level: 0
 Last Check: 2024-05-05 13:54:27
 Check Interval: 0
 Creator OS: Linux
 Revision Level: Dynamic Rev
 Default Reserved UID: 10010
 Default Reserved GID: 1065

For EXT4_DYNAMIC_REV Super Blocks Only

First Inode: 11
 Inode Size: 256

Block Group Number: 0
Compatible feature: Has Journal Extended Attributes Resize Inode Directory Index
Incompatible feature: Filetype Recover Extents 64-bit Flex_Block_Group Encrypted
Read-Only Compatible feature: Sparse_Super Large_File Huge_File Directory_NLink
Extra_ISize
UUID: 14 73 f2 7a aa 17 43 2f b0 d6 e5 b8 f3 73 e2 7d
Volume Name:
Last Mounted: /data
Algorithm Usage Bitmap: 0

EXT4_FEATURE_COMPAT_DIR_PREALLOC on for performance hints Directory
preallocation

Preallocated Blocks: 0
Preallocated Directory Blocks: 0
Reserved GDT Blocks: 1024

For EXT4_FEATURE_COMPAT_HAS_JOURNAL Only

Journal UUID: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Journal Inode: 8
Journal Device: 0
Last Orphan: 0
HTREE Hash Seed: 278549085618096283103966179399117498906
Default Hash Version: Half MD4
Reserved Char Pad: 1
Descriptor Size: 64
Default Mount Options: XATTR User ACL
First Meta Block Group: 0
MKFS Time: 2023-12-27 16:32:28

End of Superblock Parsing

Now going to the directory engineer mode which has the inode entry which causes the error in disk editor.

The engineer mode directory inode is 784897, where each block contains 8176 inodes (from superblock), and number of descriptors is 175 and the table starts at 0x1000, so the block group that we will need will be in 0x2800 and it's the start of the inode table.

This is calculated by no of inodes that a group can hold and which group will have this particular inode and linear calculation of the inode's location in the inode table with the use of inode size.

00000002800	00 00 30 00 10 00 30 00	20 00 30 00	0F 5F D5 1A	..0...0. .0.._õ.
00000002810	37 00 04 00 00 00 00 00	F6 C1 E3 EB D4 1A 42 92		7.....öÄäëÖ.B.
00000002820	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
00000002830	00 00 00 00 00 00 00 00	23 55 6A 0B 00 00 00 00		#Uj.....
00000002840	01 00 30 00 11 00 30 00	1F 02 30 00 58 77 F0 1F		..0...0...0.Xwõ.
00000002850	00 00 05 00 00 00 00 00	CB E5 00 00 F0 1F 34 48		Ëå..õ.4H
00000002860	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
00000002870	00 00 00 00 00 00 00 00	D3 B8 00 00 00 00 00 00		ó,.....
00000002880	02 00 30 00 12 00 30 00	1E 04 30 00 D4 30 F0 1F		..0...0...0.ô0õ.
00000002890	00 00 05 00 00 00 00 00	6C A9 00 00 F0 1F B6 4B		l©..õ.¶K
000000028A0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000000028B0	00 00 00 00 00 00 00 00	92 05 00 00 00 00 00 00		

So going to 0x300020000 to get to the inode and we can see it has 2 blocks of directory entries at 0x302010000 and 0x302012000 with depth 0.

00300020000	E8 41 E8 03 00 60 00 00	4A 06 6F 5F FB D9 37 66	èAè...`...J.o_û07f
00300020010	AA 51 8C 65 00 00 00 00	AC 0B 03 00 38 00 00 00	*Q.e.....7....8...
00300020020	00 10 08 00 70 01 00 00	0A F3 02 00 04 00 00 00	p....ó.....
00300020030	00 00 00 00 00 00 00 00	01 00 00 00 10 20 30 00	 0.
00300020040	01 00 00 00 05 00 00 00	12 20 30 00 00 00 00 00	 0.....
00300020050	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00300020060	00 00 00 00 AD 2C A9 19	2F 20 30 00 00 00 00 00	 ,@./ 0.....
00300020070	00 00 00 00 00 00 00 00	00 00 00 00 0F 75 00 00	
00300020080	20 00 34 69 00 E4 CC A8	08 50 A0 6E 00 00 00 00	.4i.äî".P n....
00300020090	AA 51 8C 65 08 C8 75 5D	00 00 00 00 00 00 00 00	*Q.e.Èu].....
003000200A0	00 00 02 EA 07 06 38 00	00 00 00 00 21 00 00 00	...ê..8.....!...
003000200B0	00 00 00 00 73 65 6C 69	6E 75 78 00 00 00 00 00	selinux.....
003000200C0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
003000200D0	00 00 00 00 00 00 00 00	00 00 00 00 75 3A 6F 62	u:ob
003000200E0	6A 65 63 74 5F 72 3A 64	61 74 61 5F 65 6E 67 69	ject_r:data_engi
003000200F0	6E 65 65 72 5F 66 69 6C	65 3A 73 30 00 00 00 00	neer_file:s0....
00300020100	A4 81 E8 03 4F 00 00 00	4A 06 6F 5F FB D9 37 66	α.è.O...J.o_û07f
00300020110	67 1A 8E 65 00 00 00 00	E8 03 01 00 08 00 00 00	g..e....è.....
00300020120	00 00 08 00 01 00 00 00	0A F3 01 00 04 00 00 00	ó.....
00300020130	00 00 00 00 00 00 00 00	01 00 00 00 D6 82 30 00	Ö.0.
00300020140	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00300020150	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00300020160	00 00 00 00 AE 2C A9 19	00 00 00 00 00 00 00 00	®,@.....
00300020170	00 00 00 00 00 00 00 00	00 00 00 00 CD F2 00 00	íò..
00300020180	20 00 C0 06 00 34 5B 9D	00 E4 C3 6B 00 00 00 00	.Ä..4[...äÄk....
00300020190	AA 51 8C 65 08 C8 75 5D	00 00 00 00 00 00 00 00	*Q.e.Èu].....
003000201A0	00 00 02 EA 07 06 38 00	00 00 00 00 21 00 00 00	...ê..8.....!...
003000201B0	00 00 00 00 73 65 6C 69	6E 75 78 00 00 00 00 00	selinux.....
003000201C0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

Looking for the files in it, we can find several files with inode 0, which should be the main reason for causing the segmentation fault

00302012800	2E 74 78 74 65 FB 0B 00	F0 07 1F 01 70 65 72 73	.txtedü..ð...pers
00302012810	69 73 74 70 61 74 68 5F	2D 31 38 35 34 36 5F 2D	istpath_-18546_-
00302012820	4A 50 5F 2D 43 50 57 2E	74 78 74 55 53 5F 50 52	JP_-CPW.txtUS_PR
00302012830	45 50 41 49 44 2E 74 78	74 6D 6F 6E 00 00 00 00	EPAID.txtmon...0
00302012840	2C 00 22 01 70 65 72 73	69 73 74 70 61 74 68 5F	,..".persistpath_
00302012850	2D 31 38 35 34 33 5F 2D	4A 50 5F 2D 54 45 4C 43	-18543_-JP_-TELC
00302012860	45 4C 2E 74 78 74 2E 74	D1 FA 0B 00 2C 00 21 01	EL.txt.tÑú...!..
00302012870	70 65 72 73 69 73 74 6E	61 6D 65 5F 2D 31 38 35	persistname_-185
00302012880	34 35 5F 2D 4A 50 5F 2D	4F 50 54 55 53 2E 74 78	45_-JP_-OPTUS.tx
00302012890	74 6D 6F 6E 00 00 00 00	34 00 2B 01 70 65 72 73	tmon....4.+.pers
003020128A0	69 73 74 70 61 74 68 5F	2D 31 38 35 34 36 5F 2D	istpath_-18546_-
003020128B0	4A 50 5F 2D 56 4F 44 41	46 4F 4E 45 5F 4E 4F 4E	JP_-VODAFONE_NON
003020128C0	45 45 41 2E 74 78 74 61	00 00 00 00 34 00 29 01	EEA.txta....4.).
003020128D0	70 65 72 73 69 73 74 70	61 74 68 5F 2D 31 38 33	persistpath_-183
003020128E0	32 36 5F 2D 4A 50 5F 2D	4F 50 54 55 53 5F 50 52	26_-JP_-OPTUS_PR
003020128F0	45 50 41 49 44 2E 74 78	74 4E 5F 63 DC FA 0B 00	EPAID.txtN_cÜü..
00302012900	2C 00 22 01 70 65 72 73	69 73 74 70 61 74 68 5F	,..".persistpath_
00302012910	2D 31 38 33 32 32 5F 2D	4A 50 5F 2D 54 45 4C 43	-18322_-JP_-TELC
00302012920	45 4C 2E 74 78 74 5F 63	00 00 00 00 2C 00 22 01	EL.txt_c.....".

So the inode is unallocated and that is what is breaking the tool.

So let's make an if statement and that should fix our segmentation fault.

```
if(fs_file->meta == NULL)
{
    tsk_print_sanitized(hFile, fs_file->name->name);
    // tsk_fprintf(hFile, "\n");
    return;
}
```

```
439      /* print a * if it is deleted */
440      if (fs_file->name->flags & TSK_FS_NAME_FLAG_UNALLOC)
441          tsk_fprintf(hFile, "* ");
442      // printf("[Azr43lKn1ght %p]",fs_file->meta);
443
444      if(fs_file->meta == NULL)
445      {
446          tsk_print_sanitized(hFile, fs_file->name->name);
447          // tsk_fprintf(hFile, "\n");
448          return;
449      }
450
```

Now let's compile and run the tool

```

+ r/r 785176: persistname_-18325_-JP_-CPW.txt
+ r/r 785180: persistname_-18547_-JP_-ATT.txt
+ r/r 785181: persistname_-18545_-JP_-VODAFONE_EEA.txt
+ r/r 785185: persistpath_-18546_-JP_-OPTUS_POSTPAID.txt
+ r/r 785188: persistname_-19621_-JP_-VODAFONE_EEA.txt
+ r/r 785189: persistpath_-18322_-JP_-ATT.txt
+ r/r 785194: persistname_-18547_-JP_-OPTUS_PREPAID.txt
+ r/r 785201: persistpath_-18543_-JP_-CPW.txt
+ r/r 785210: persistname_common_EUEX_-OPTUS_PREPAID.txt
+ r/r 785220: persistname_-18542_-JP_-OPTUS.txt
+ r/r 785221: persistpath_-18321_-JP_-OPTUS_PREPAID.txt
+ r/r 785242: persistname_-18326_-JP_-VODAFONE_EEA.txt
+ r/r 785253: persistpath_-18546_-JP_-CPW.txt
+ r/- * persistpath_-18543_-JP_-TELCEL.txt
+ r/- * persistpath_-18546_-JP_-VODAFONE_NONEEA.txt
+ r/- * persistpath_-18326_-JP_-OPTUS_PREPAID.txt
+ r/- * persistname_-18540_-JP_-TELCEL.txt
+ r/- * persistname_-18546_-JP_-VODAFONE_NONEEA.txt
+ r/- * persistpath_-18543_-JP_-ATT.txt
+ r/- * persistpath_-18321_-JP_-OPTUS_POSTPAID.txt
+ r/- * persistpath_-18542_-JP_-TELCEL.txt
+ r/- * persistpath_-18541_-JP_-VODAFONE_NONEEA.txt
+ r/- * persistpath_-18326_-JP_-VODAFONE_NONEEA.txt
+ r/- * persistpath_-18325_-JP_-OPTUS_POSTPAID.txt
+ r/- * persistpath_-19621_-JP_-OPTUS_POSTPAID.txt
+ r/- * persistname_-18321_-JP_-VODAFONE_NONEEA.txt
+ r/- * persistpath_-18321_-JP_-VODAFONE_EEA.txt
+ r/- * persistpath_-18321_-JP_-CPW.txt
+ r/- * persistname_-18326_-JP_-OPTUS_PREPAID.txt
+ r/- * persistpath_-18542_-JP_-CPW.txt
+ r/- * persistpath_-18542_-JP_-VODAFONE_EEA.txt
+ r/- * persistpath_-18545_-JP_-CPW.txt
+ r/- * persistname_-18541_-JP_-TELCEL.txt
+ r/r 784898: .sd.txt
+ r/r 784902: appchannel_common_BD_common.txt
+ r/r 785001: persistpath_common_MM_common.txt
+ r/r 785006: persistpath_common_PK_common.txt

```

Now there is no segmentation fault and the code is fixed, let's try running fbekeyfind on it.

```

azr43lkn1ght@Azr43lkn1ght: /mnt/e/fbekeyfind/fbekeyfind$ python3.8 findMasterKeys.py --part ../../userdata/realme.dsk ../../dump_dram_0x40000000.bin
Extracting Encryption Attributes | 0/14130
Traceback (most recent call last):
  File "findMasterKeys.py", line 386, in <module>
    g_attributes, g_inodes = g_fls.get_all_encryption_attributes_from_fs()
  File "findMasterKeys.py", line 53, in get_all_encryption_attributes_from_fs
    attr = self.extract_encryption_xattr_istat(i)
  File "findMasterKeys.py", line 128, in extract_encryption_xattr_istat
    _, _, _, content_mode, _, _, _ = line.split()
ValueError: not enough values to unpack (expected 7, got 5)

```

But we still have an error, so we will also fix it and also looking into the regex, it was perfect for filtering all the allocated inodes

```

120
121     output = output.decode('utf-8')
122     content_mode = None
123     name_mode = None
124     key_desc_str = None
125     nonce_desc_str = None
126     for line in output.splitlines():
127         if line.startswith("FBE Content Mode"):
128             # print(line)
129             try:
130                 _, _, _, content_mode, _, _, _ = line.split()
131             except:
132                 _, _, _, content_mode, _ = line.split()
133                 content_mode = int(content_mode)
134                 continue
135         if line.startswith('FBE Name Mode'):
136             _, _, _, name_mode, _, _, _ = line.split()
137             name_mode = int(name_mode)
138             continue
139         if line.startswith('FBE Key Descriptor'):
140             key_desc = line.split()
141             key_desc = key_desc[3:]
142             key_desc_str = list()
143             for element in key_desc:
144                 key_desc_str.append(int(element, 16))
145             continue

```

Fixing this segment of the code, we are able to run the tool.

```

azr43lkn1ght@Azr43lkn1ght:/mnt/d/Android-FBE/fbekeyfind$ python3.8 findMasterKeys.py --part /mnt/e/userdatarealm.dsk /mnt/e/dump_dram_0x40000000.bin
Extracting Encryption Attributes | | 262/14130
Extracting Encryption Attributes |#####| 14130/14130
Found 13157 Files with Encryption Attributes in Image /mnt/e/userdatarealm.dsk
Using aeskeyfind
Found 484 AES-256-Bit Keys in Memory Dump /mnt/e/dump_dram_0x40000000.bin
0 of 13157 have unexpected encryption mode
Cracking |#####| 6275889/6275889
Result for Master Key Descriptor: 8bff6608f979c3a3
Found Keys: 23da98d7147e0cb94ca5410cf54d618d08967bb40dbd4ebc1ef759e8fbacfe1e, 988fe571995d4ad99004663b1acc0c947c1a4883f300eee5b25829ce51882679
With Hits: 257, 1, 1, 1, 1, 1, 1, 1, 1
Result for Master Key Descriptor: 735b1d55cfc20286
Found Keys: e5ec67806054cd9b6724ab9a65c809b36ae51c0ded072c5d442719664a5990b6, 51d67eacc4ced98207ac9fcc96aad903e71e201f3ac61f68fdac030c6449295d
With Hits: 48, 1, 1, 1, 1, 1, 1, 1, 1
Result for Master Key Descriptor: 7f794cc2ec5e1855
Found Keys: 26af0d392b1aa1c4281bdd4604df936b9de58ae71915724a11f3b624f0d43c82, de20b76a916fd98adcd0273143544a05cdd0c672447e53d751df75c206ca9ee1
With Hits: 73, 1, 1, 1, 1, 1, 1, 1, 1

```

And the tool works, now let's try if the files are decrypted with the key and how was it before


```

azr43lkn1ght@Azr43lKn1ght:/mnt/d/Android-FBE/fbekeyfind$ fls /mnt/e/userdatarealme.dsk 523302
d/d $ 523666:   d4 Y^49 ^!z 
d/d $ 524083:  c    yW84    M 
d/d $ 654279:     i   5I   W*
d/d $ 654280:  F  N D          
d/d $ 654281: sXNb4n{        q
d/d $ 654283: 4a9    X'./ =X'  
d/d $ 654285:     ,  5 7  L    
d/d $ 654286:     = N    [      
d/d $ 654287: 4D    U  7    ; W
d/d $ 654288: k  b  ,Nak'  g'      
d/d $ 524208: ' vQ'  U   ]    
d/d $ 524403:   P                
r/r $ 524452: ?; I*                
r/r $ 524202: -^0' |  |'- V 2    
d/d $ 525195: R\      xj~r-h;M
r/r $ 525748: a'  $A   5(  B  
r/d * $ 525771(realloc): m+  hhN  =    F    
azr43lkn1ght@Azr43lKn1ght:/mnt/d/Android-FBE/fbekeyfind$ fls -K fbe.keys /mnt/e/userdatarealme.dsk 523302
d/d $ 523666: Android
d/d $ 524083: DCIM
d/d $ 654279: Pictures
d/d $ 654280: Music
d/d $ 654281: Podcasts
d/d $ 654283: Ringtones
d/d $ 654285: Alarms
d/d $ 654286: Notifications
d/d $ 654287: Movies
d/d $ 654288: Download
d/d $ 524208: .mcs
d/d $ 524403: ColorOS
r/r $ 524452: .ext4
r/r $ 524202: .sstmp
d/d $ 525195: mtkloq

```

Let's look into how it is decrypted, let's take inode 523666 as an example

It's present in the block group at 0x2000 and the inode table is located in 0x200020000 and our inode is located in 0x200039100.

Looking into the extended attributes, we will be able to get the encryption method for file as well as content, key descriptor and file nonce for DEK derivation.

00200039100	FD 41 FF 03 00 10 00 00	D8 27 0C 5E D9 27 0C 5E	ýÄÿ.....ø'.^û'.^
00200039110	D9 27 0C 5E 00 00 00 00	FF 03 04 00 10 00 00 00	Û'.^.....ÿ.....
00200039120	00 08 08 00 03 00 00 00	0A F3 01 00 04 00 00 00	ó.....
00200039130	00 00 00 00 00 00 00 00	01 00 00 00 33 22 20 00	3" .
00200039140	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00200039150	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00200039160	00 00 00 00 8A F5 51 66	32 22 20 00 00 00 00 00	öQf2" .
00200039170	00 00 00 00 00 00 00 00	00 00 00 00 DF 5B 00 00	ß[.
00200039180	20 00 60 B9 00 D9 6A CA	00 D9 6A CA 00 A9 6B 63	.^'.ÛjÊ.ÛjÊ.©kc
00200039190	D8 27 0C 5E 00 A9 6B 63	00 00 00 00 00 00 00 00	ø'.^'.©kc.....
002000391A0	00 00 02 EA 01 09 40 00	00 00 00 00 1C 00 00 00	...ê..@.....
002000391B0	00 00 00 00 63 00 00 00	0C 01 3C 00 00 00 00 00	c.....<.....
002000391C0	01 00 00 00 00 00 00 00	73 64 63 61 72 64 66 73	sdcardfs
002000391D0	2E 75 69 64 00 00 00 00	00 00 00 00 00 00 00 00	.uid.....
002000391E0	30 00 00 00 01 7F 04 00	8B FF 66 08 F9 79 CB A3	0.....ÿf.ÛÿÊÊ
002000391F0	3B 8A D3 50 53 A7 AA 2B	46 FE AE 0C 38 C7 FD 59	;.óPSS^+Fb@.8Cÿÿ

Here we get the nonce in the highlighted last line and key descriptor that is 8 bytes before it and content and file encryption as 7F(127 Private) and 4(AES-CTS).

```

azr43lkn1ght@Azr43lKn1ght:/mnt/e$ istat userdatarealme.dsk 523666
inode: 523666
Allocated
Group: 64
Generation Id: 1716647306
uid / gid: 1023 / 1023
mode: drwxrwxr-x
Flags: Compression Error, Extents,
size: 4096
num of links: 4

Extended Attributes (Block: 2105906)
security.selinux=u:object_r:media_rw_data_file:s0

Extended Attributes (Inode Included)
FBE Content Mode: 127 (PRIVATE)
FBE Name Mode: 4 (AES 256 CTS)
FBE Key Descriptor: 8B FF 66 08 F9 79 CB A3
FBE Nonce: 3B 8A D3 50 53 A7 AA 2B 46 FE AE 0C 38 C7 FD 59
user.sdcardfs.uid=0

Inode Times:
Accessed:      2020-01-01 05:02:16.417000000 (UTC)
File Modified: 2020-01-01 05:02:17.849000000 (UTC)
Inode Modified: 2020-01-01 05:02:17.849000000 (UTC)
File Created:  2020-01-01 05:02:16.417000000 (UTC)

Direct Blocks:
2105907

```

Which is the same as what istat gives.

And the corresponding masterkey is

```
8bff6608f979cba3:23da98d7147e0cb94ca5410cf54d618d08967bb40dbd4ebc1ef759e8fba
cfe1e:988fe571995d4ad99004663b1acc0c947c1a4883f300eee5b25829ce51882679
```

And with which we can use KDF and get the DEK as

```
73c26a24dcb2ed2d9cdbd03c6421a683c73e90c0d2f42dcd0ccfc504ff3a1c4ab701e28c9db59
e2d32a3cff221f91112c91a06bfd5412405617112483b60650b3142e5623473e7b8be81edbf
a0612760
```

And now we can use AES-CTS on the name and decrypt it and we get it as Android

So the whole problem is completely fixed and the problem on the forensic tool's error/bug is solved.

The whole Fixed tool can be downloaded from :

https://drive.google.com/file/d/1oSqmiHDxTf8eAxkU0wxVKJ_sDb-5DP4N/view?usp=sharing

And we are not adding any Test Data of Android device as the tool whole works in all generic Android Device Capture by the same way and unallocated inode can be edited manually in the file system and checked too.

Original Research and Tool Link:

<https://www.cs1.tf.fau.de/research/system-security-group/one-key-to-rule/>

Summary:

Fbekeyfind and Sleuthkit extended with FBE support made by Tobias Gross and team is the only tool that is available to decrypt as well as recover masterkeys for FBE encrypted modern android with cold boot attack. Because of the segmentation fault, the whole tool will become useless as everything depends on the masterkey recovery. We were able to fix it and completely fix it and we can also edit the KDF change from ECB to AES HEH for a few versions for better support. We also have the EXT4 parser to analyze the whole ext4 structure easily and efficiently. With the fix of the tool, we tested the error fix and it works as intended and keys are recovered and tested for decryption. With this anyone in the community who uses it on an actual mobile device will be benefited.